烟幕干扰弹投放策略的时空协同优化模型

摘　　要

随着无人机技术的飞速发展和精确制导武器的广泛应用，发展更先进、更有效的保护手段具有其重要性。烟幕干扰作为一种有效的无源干扰手段，能够阻断侦察设备的视线，有效干扰敌方制导系统，从而保护我方目标。本文针对无人机投放烟幕干扰弹对来袭导弹的干扰问题，建立了基于视线阻断的烟幕遮蔽有效性模型，并采用多种智能优化算法求解最优投放策略。

针对问题一无人机 FY1 投放单枚烟幕干扰弹对导弹 M1 的干扰问题，在给定无人机飞行方向、飞行速度、烟幕弹投放点与起爆点的条件下，建立了**基于视线阻断的烟幕遮蔽有效性计算模型**。通过高密度采样真目标表面与内部点，结合烟幕球体与导弹-目标视线的几何关系，判断每一时刻是否完全遮蔽。数值模拟结果表明，在给定参数下，总有效遮蔽时长约为 1.39 秒。

针对问题二无人机 FY1 投放单枚干扰弹对导弹 M1 的最大化遮蔽问题，构建以无人机飞行方向、飞行速度、烟幕投放延迟和起爆延迟为决策变量的四维优化模型。采用**遗传算法**进行全局优化，通过实数编码、锦标赛选择、模拟二进制交叉和多项式变异等操作，最大化有效遮蔽时间。优化结果表明，最大有效遮蔽时长约为 4.66 秒。

针对问题三单架无人机投放三枚烟幕弹对导弹 M1 的干扰问题，建立了包含 8 个决策变量的优化模型。采用**遗传算法**、**粒子群算法**和**模拟退火**的三阶段混合优化策略，在满足投放间隔约束下最大化遮蔽时间。优化结果显示，三枚烟幕弹协同可实现总有效遮蔽时长约 6.39 秒。

针对问题四无人机 FY1、FY2、FY3 对导弹 M1 的干扰问题，建立了包含 12 个决策变量的优化模型。采用**遗传算法**、非线性优化、全局搜索和并行运算的混合优化策略，在满足投放间隔的约束下最大化遮蔽时间。优化结果显示，三架无人机投放烟幕弹可实现总有效遮蔽时长约 14.20 秒。

针对问题五五架无人机各至多投放三枚烟幕干扰弹对三枚导弹的干扰问题，建立了更高维的优化模型，通过**迭代优化**和**粒子群算法**，在满足投放间隔的约束下最大化遮蔽时间。优化结果显示，可实现总有效遮蔽时长约 27.3 秒。

关键词： 烟雾遮蔽模型 视线阻断 遗传算法 粒子群算法 优化模型

一、问题重述

1.1 问题背景

烟幕干扰是通过在空气中施放大量气溶胶微粒，在大气中特定区域形成遮蔽，掩盖要保护的目标。对烟幕干扰弹进行定点定时投放，并采用时间引信控制起爆时序，能够使其达到指定位置，精准干扰敌方导弹，以保护我方目标。

1.2 问题要求

无人机投放烟幕干扰弹在导弹和真目标之间形成烟幕遮蔽，从而保护真目标。无人机携烟幕干扰弹做等高匀速直线运动，烟幕弹被投放后做平抛运动，后经起爆匀速下沉，瞬时形成一个球体云团为真目标提供遮蔽。为掩护真目标，还存在一个假目标，导弹来袭方向直指假目标。题目给出 3 枚导弹（M1-3）和 5 架无人机（FY1-5）的初始位置信息。

需针对不同情况，设计如何投放烟幕干扰弹，包括确定无人机的飞行方向、速度与烟幕干扰弹的投放点、起爆点，最终使得多枚烟幕干扰弹对真目标的有效遮蔽时间尽可能长。

问题 1　无人机 FY1 投放 1 枚烟幕弹实施对 M1 的干扰，给定烟幕弹投放策略，需计算其对 M1 的有效遮蔽时长。

问题 2 至 5 均需确定烟幕弹投放策略，使得有效遮蔽时间尽可能长。分别为以下不同情况：

问题 2　无人机 FY1 投放 1 枚烟幕弹实施对 M1 的干扰；

问题 3　无人机 FY1 投放 3 枚烟幕弹实施对 M1 的干扰；

问题 4　3 架无人机各投放 1 枚烟幕弹实施对 M1 的干扰；

问题 5　5 架无人机投放至多 3 枚烟幕弹实施对 3 枚导弹的干扰。

二、模型假设

1.重力加速度 $g=9.8m/s^2$。

2.烟幕遮蔽判断基于导弹对真目标的视线是否被烟幕云团阻挡。

3.警戒雷达发现来袭导弹时，立即指派任务给无人机。无人机在初始位置瞬时改变飞行方向与飞行速度，后续不再改变方向与速度。

4.不考虑空气阻力，烟幕干扰弹脱离无人机至起爆前在重力作用下作理想的平抛运动。

5.烟幕干扰弹起爆后瞬时形成球状云团。

6.烟幕云团采用特定技术，以 $3m/s$ 的速度匀速下沉，且方向始终垂直向下。

7.无人机投放烟幕干扰弹后不再影响烟幕弹的飞行状态。

8.空地导弹[3]始终朝向假目标以 $300m/s$ 匀速直线飞行，不会更改路径。

9.烟幕云团之间无相互作用。

三、符号说明

符号	说明	单位
X_{ij}	无人机 FYi 所投放的第 j 枚烟雾干扰弹	
$u=(\cos\theta,\sin\theta,0)$	无人机飞行方向	
v	无人机飞行速度	m/s
$\overrightarrow{U_i}$	无人机 FYi 初始位置	
P_{drop}	烟雾干扰弹投放点坐标	
P_{det}	烟雾干扰弹起爆点坐标	
t_{drop}	投放延迟时间	s
t_{det}	起爆延迟时间	s
$\overrightarrow{M_i}, i=1,2,3$	导弹 Mi 初始位置	
v_m	导弹飞行速度	m/s
$\overrightarrow{d_m}$	导弹飞行方向单位向量	
$\{Q_i\}, i=1,2,...,N$	真目标采样点集合	
$M_i(t)$	t 时刻导弹位置	
$C(t)$	t 时刻烟幕中心位置	
T_{shield}	累计有效遮蔽时间	s

四、模型建立与求解

4.1 问题一：变量固定时单枚烟幕弹有效遮蔽时长

根据第一问所给数据，我们可以得到一架无人机 FY1，一枚烟幕弹，一颗导弹 M1,三者的运动状态已知。我们根据烟幕干扰弹和导弹的运动状态，确定烟幕干扰弹对 M1 的有效遮蔽时长。具体分析如下：

4.1.1 问题分析

题目中需设计烟幕干扰弹的投放策略，包含 4 个决策变量：无人机飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点。烟雾干扰弹 X_{11} 的投放与起爆包括可简化为三个阶段，在第一阶段中，无人机受领任务后携烟幕弹瞬时调整方向速度，后等高匀速直线飞行；第二阶段中，烟幕弹脱离无人机，在重力作用下做平抛运动；第三阶段，烟幕弹起爆，瞬时形成球状烟幕云团，云团以 3m/s 的速度匀速下沉。

问题一要求利用无人机 FY1 投放 1 枚烟幕干扰弹对导弹 M1 实施干扰，在给定无人机的飞行方向及速度、烟幕干扰弹的投放点及起爆点的情况下，计算总有效遮蔽时

长。此为四个决策变量都确定的情况。

我们通过建立基于视线阻断的烟幕遮蔽有效性计算模型，在圆柱体表面和内部生成高密度采样点，在时空两个维度，对烟幕干扰弹是否有效遮蔽了真目标进行判断。

4.1.2 基于视线阻断的烟幕遮蔽有效性计算模型

由于真目标是一个柱体，烟幕弹起爆后的云团球体（简称为烟幕球体）对导弹产生有效遮盖的几何条件应为：导弹到真目标上任意一点的连线（线段），均与烟幕球体存在交点。

导弹位置 $\overrightarrow{M_i}$ 到真目标上任意一点 $\vec{Q}_i$，烟幕球球心为 $\vec{C}$，半径为 $R(R=10)$。线段 M_iP_i 上的任意点可表示为 $\overrightarrow{M_i}+s\times\left(\vec{P_i}-\overrightarrow{M_i}\right)$，其中 $s\in[0,1]$。该点在球面上时，到球心距离满足：$\left\|\vec{P_i}+s\cdot\left(\vec{P_i}-\overrightarrow{M_i}\right)-\vec{C}\right\|=R$

展开后可转化为关于 s 的二次方程：

$$a\times s^2+b\times s+c=0$$

其中 $a=\left\|\vec{P_i}-\overrightarrow{M_i}\right\|^2$，$b=-2\times\left(\vec{C}-\overrightarrow{M_i}\right)\times\left(\vec{P_i}-\overrightarrow{M_i}\right)$，$c=\left\|\vec{C}-\overrightarrow{M_i}\right\|^2-R^2$。

关于 s 的二次方程若存在 $s\in[0,1]$ 的解，则线段 M_iP_i 上存在某点落在烟幕球面或烟幕球内。即对于判别式 $\triangle=b^2-4ac$，若 $\triangle\geq 0$，则存在两个实根 s_1 和 s_2 $(s_1\leq s_2)$，若与此同时满足 $s_2\geq 0$ 且 $s_1\leq 1$，则线段与球相交。

MATLAB 代码通过生成真目标的高密度采样点，对每个时刻遍历所有采样点是否满足条件，只有所有点均满足条件时，认为烟幕球对导弹产生有效遮蔽。

采样点包括：底面和顶面圆周点、侧面点、内部网格点、轴线点。确保覆盖整个目标体积，避免漏判。

4.1.3 基于烟幕遮蔽有效性模型的问题一求解

Ⅰ 烟幕弹位置

建立三维直角坐标系，假目标为原点 $(0,0,0)$，真目标位于 $(0,200,0)$。所有运动均在重力场中考虑。无人机 FY1 以 120m/s 等高匀速直线飞向假目标，水平方向向量为 $\overrightarrow{d_{xy}}=\dfrac{\vec{O}(1:2)-\overrightarrow{U_1}(1:2)}{\left\|\vec{O}(1:2)-\overrightarrow{U_1}(1:2)\right\|}=\dfrac{(-17800,0)}{17800}=(-1,0)$，故烟幕弹投放位置为 $\overrightarrow{P_{drop}}=(17620,0,1800)$。

投放后到起爆前烟幕弹做平抛运动，

$z_{det}=z_{drop}-\dfrac{1}{2}gt_{det}^2=1800-\dfrac{1}{2}\times 9.8\times(3.6)^2=1736.496$，

$\overrightarrow{P_{drop_xy}}+v\times t_{det}\times\overrightarrow{d_{xy}}=(17620,0)+120\times 3.6\times(-1,0)=(17188,0)$，故起爆位置为 $P_{det}=(17188,0,1736.496)$。

烟幕中心 C 在时间 t 的位置为

$$P(t)=(17188,0,1736.496-3\times(t-5.1))。$$

Ⅱ 导弹位置

导弹 M1 从初始位置 $\overrightarrow{M_1}$ 以 300 m/s 匀速直线飞向假目标 $\vec{O}$，其飞行方向为 $\overrightarrow{d_m}=\dfrac{-(20000,0,2000)}{\sqrt{20000^2+2000^2}}\approx(-0.995,0,-0.100)$，在时间 t 的位置为 $\overrightarrow{M_1}(t)=\overrightarrow{M_1}+v_m\times t\times\overrightarrow{d_m}$。

Ⅲ 遮蔽判断

为准确评估遮蔽效果，对真目标进行密集采样：

采样位置类型	采样点分布及数量
底面圆周	60 个均匀分布点
顶面圆周	60 个均匀分布点
侧面	20 层高度，每层 60 个点
内部空间	5 个半径层次，10 个高度层次，12 个角度
轴线关键点	底面圆心、1/4 高度、1/2 高度、3/4 高度、顶面圆心

去除重复点后，共得到 2581 个采样点，确保能够全面反映目标的几何特征。

取时间步长 $\triangle = 0.01s$，取时间范围 $[5.1, 25.1]$，运用基于视线阻断的烟幕遮蔽有效性计算模型，只有当所有 2581 个采样点的视线同时被阻挡时，才认为目标被完全遮蔽。

Ⅳ 求解结果

通过数值计算，得到以下结果：

总有效遮蔽时长：1.3900 秒

4.2 问题二：单架无人机投放单枚烟幕干扰弹对导弹 M1 的干扰

根据第二问所给数据，我们可以得到一架无人机 FY1,一颗导弹 M1，两者的空间位置已知，一枚烟幕弹的速度也已知。我们根据 FY1 和 M1 的位置坐标，确定无人机 FY1 的飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点，使得遮蔽时间尽可能长。具体分析如下：

4.2.1 问题分析

第二问要求确定无人机 FY1 的飞行方向、飞行速度、烟幕弹投放点和起爆点，使烟幕球对真目标的有效遮蔽时间尽可能长。关键是需要优化四个参数：飞行角度 θ、飞行速度 v，投放延迟 t_{drop} 和起爆延迟 t_{det}。因为需要同时在连续解空间中确定无人机的飞行参数和烟幕弹的时序参数，使用遗传算法进行全局优化求解。

4.2.2 遮蔽判断

4.2.1 中四个参数与决策变量相关，无人机的飞行角度 θ 决定飞行方向 $u = (\cos\theta, \sin\theta, 0)$，烟幕弹投放点位置为 $P_{drop} = \overrightarrow{U_1} + v \cdot t_{drop} \cdot u$，起爆点位置为 $P_{det} = P_{drop} + v \cdot t_{det} \cdot u - \left(0, 0, \frac{1}{2} g t_{det}^2\right)$。

t 时刻，烟幕球体中心位置为 $S(t) = \left(P_{det,x}, P_{det,y}, P_{det,z} - 3 \cdot \left(t - t_{drop} - t_{det}\right)\right)$，导弹位置为 $\overrightarrow{M_1}(t) = (20000, 0, 2000) + 300 \cdot t \cdot \overrightarrow{d_m}$。

此问中，同样对真目标进行采样，生成高密度采样点覆盖圆柱体表面和内部（包括底面圆周、顶面圆周、侧面和内部网格点，数量在 1000 以上），以确保遮蔽判断的准

确性。

运用 4.1.2 的烟幕遮蔽有效性模型，对每个时间点 t，检查从导弹位置 $\overrightarrow{M_1}(t)$ 到每个采样点 P 的线段是否与烟幕球相交。如果所有采样点都被遮蔽，则时间 t 有效。

4.2.3 基于遗传算法的优化求解过程

（一）决策变量

建立四维决策变量向量

$$x = T_{shield} = \left[\theta, v, t_{drop}, t_{det}\right]^t$$

其中参数取值范围：$\theta \in [0, 2\pi]$，

$v \in [70, 140]$，

$t_{drop} \in [0, 60]$，

$t_{det} \in [0, 10]$。

（二）目标函数

目标函数为最大化有效遮蔽时间

$$\max T_{shield}$$

（三）遗传算法

Ⅰ 染色体编码

采用实数编码方式，染色体 $-T_{shield}$ 包含 4 个基因：$\theta, v, t_{drop}, t_{det}$。

Ⅱ 初始群体设计

种群大小选择 40 个个体。

采用均匀随机初始化策略，在每个变量的合法取值范围内生成初始种群个体：

$$x_i^{(0)} = lb_i + (ub_i - lb_i) \cdot rand(0,1), i = 1, 2, 3, 4$$

Ⅲ 适应度函数

适应度函数计算给定无人机参数下的有效遮蔽时间，由于遗传算法默认计算最小化，适应度函数为有效遮蔽时间的负值，即：

$$f_{fitness}(x) = -T_{shield}(x)$$

其中使用部分采样点而非全部 2581 个采样点。目标区域的采样点具有空间相关性，适量采样点足以反映整体遮蔽状态。通过实验验证，50 个均匀分布的采样点可以达到 95%以上的精度。

Ⅳ 遗传操作设计

1. 选择操作

采用锦标赛选择策略，从种群中随机选择若干个体，选择其中适应度最好的个体进入下一代。

2. 交叉操作

采用模拟二进制交叉（SBX），保持种群多样性：

$$x_{new} = \frac{1}{2}\left[(1+\beta)x_1 + (1-\beta)x_2\right]$$

其中 β 为交叉分布参数。

3. 变异操作

采用多项式变异，以一定概率随机改变基因值：

$$x_{mut} = x + \delta \cdot (ub - lb)$$

其中δ为变异幅度。

Ⅴ 控制参数设定

1. 算法和遗传操作参数

参数	取值	说明
种群大小	40	平衡计算效率和多样性
最大进化代数	30	基于收敛性分析设定
交叉概率	0.8	默认值
变异概率	0.1	默认值
锦标赛选择大小	2	tournamentSize 参数
分布指数（交叉）	20	SBX 交叉分布参数
分布指数（变异）	20	多项式变异分布参数

2. 终止条件

$$\begin{cases} \text{最大代数:30代} \\ \text{函数容差：连续5代适应度变化小于}10^{-6} \\ \text{时间限制：最长运行时间3600秒} \\ \text{代际停滞:10代无改进} \end{cases}$$

4.2.4 问题二的求解结果

通过遗传算法的优化（图 1 遗传算法收敛曲线图），最大遮蔽时间为：

4.6600s

对应的最优参数为：$\begin{cases} \theta = 0.1251rad\left(7.17^{\circ}\right) \\ v = 94.9908m/s \\ t_{drop} = 0.1507s \\ t_{det} = 0.8355s \end{cases}$

投放点坐标为：$(17814.2003,1.7861,1800.0000)$

起爆点坐标为：$(17892.7546,11.6666,1796.5960)$

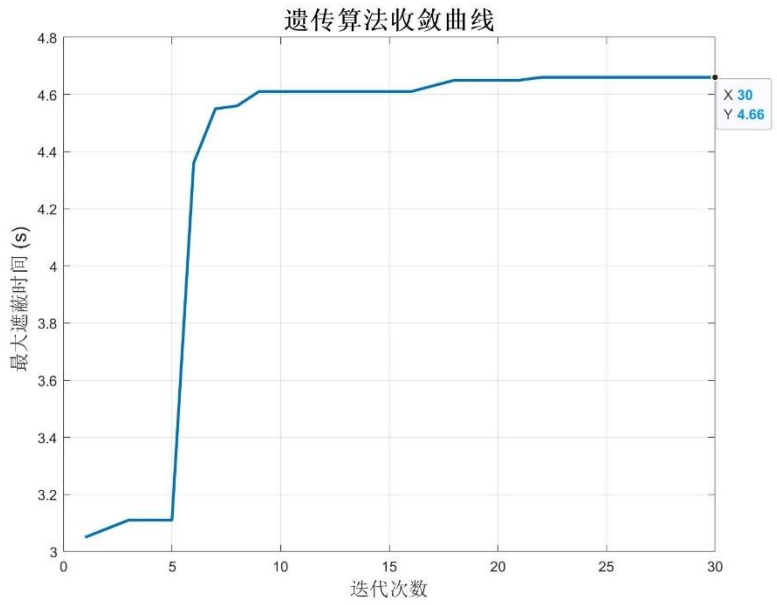


图 1 遗传算法收敛曲线图

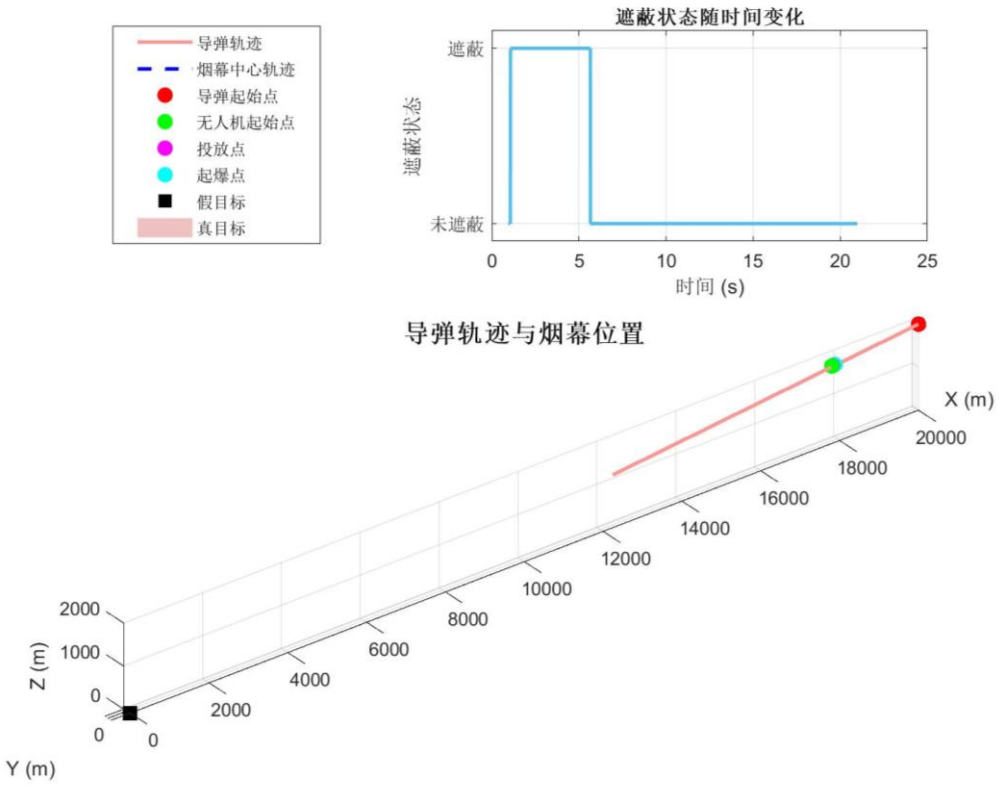


图 2 轨迹与屏蔽状态图示[3]

4.3 问题三：单架无人机投放三枚烟幕干扰弹对导弹 M1 的干扰

根据第三问所给数据，我们可以得到一架无人机 FY1,一颗导弹 M1，两者的空间位置已知，三枚烟幕弹的速度也已知。我们根据 FY1 和 M1 的位置坐标，确定烟幕干扰弹的投放策略，使得遮蔽时间尽可能长。具体分析如下：

4.3.1 问题分析

问题三在问题二的基础上，无人机 FY1 变为投放 3 枚烟幕干扰弹，需要确定烟幕弹投放策略，使得对导弹 M1 的有效遮蔽时间最大化。

4.3.2 建模过程

决策变量的参数相较于问题二，增加为 8 个：

θ(rad)：无人机飞行方向
v(m/s)：无人机飞行速度
$t_{drop_1}, t_{drop_2}, t_{drop_3}$ (s)：三枚烟幕弹延迟投放时间
$t_{det_1}, t_{det_2}, t_{det_3}$：三枚烟幕弹延迟起爆时间

决策变量：

$$x = T_{shield} = \left[\theta, v, t_{drop_1}, t_{drop_2}, t_{drop_3}, t_{det_1}, t_{det_2}, t_{det_3}\right]^t$$

三枚烟幕弹投放点与起爆点的位置也相应为：

$$P_{drop_i} = \overrightarrow{U_1} + v \cdot t_{drop_i} \cdot u$$

$$P_{det_i} = P_{drop_i} + v \cdot t_{det_i} \cdot u - \left(0, 0, \frac{1}{2} g t_{det_i}^2\right)$$

在各自起爆后，烟幕团中心随时间下沉：$C_i(t) = P_{det_i} - \left(0, 0, 3 \cdot \left(t - t_{b_i}\right)\right)$，其中 $t_{b_i} = t_{drop_i} + t_{det_i}$。

仍沿用 4.1.2 的模型判断烟幕遮蔽的有效性。

新增约束条件：

三枚烟幕弹两两之间投放时间至少间隔 1 s，即存在投放时间约束：

$$s.t. \begin{cases} t_{drop_2} \geq t_{drop_1} + 1 \\ t_{drop_3} \geq t_{drop_2} + 1 \end{cases}$$

目标函数：

$$\max \sum_{t \in T} \Delta t \cdot \text{II}\left(\text{目标在时间} t \text{被遮蔽}\right)$$

其中 II 为指示函数，括号内条件成立，则函数值为 1，反之为 0。

4.3.3 基于遗传算法、粒子群算法和模拟退火的优化求解过程

基于问题二，用遗传算法（GA）进行全局搜索，再用粒子群算法（PSO），以前者结果为初始点，进行局部精细搜索，利用个体和群体经验快速收敛到优质区域，在后期使用模拟退火，得到全局最优解。（图 3）

Ⅰ 遗传算法

在 8 维参数空间中进行广域搜索，通过选择、交叉、变异的操作机制不断调整，直到满足终止条件。使用全局算法能有效地避免陷入局部最优的情况，并为下一步提供高质量初始点。

Ⅱ 粒子群算法

粒子群算法（Particle swarm optimization,PSO）[1][3]是一种由 Kennedy 和 Eberhart 在鸟群捕食与鱼类觅食行为的研究中获得灵感后提出的一种群体智能算法，是一种基于迭代模式的优化算法。

问题三中，每个粒子为一个完整的烟幕弹投放策略方案，包含 8 个决策变量，是一个 8 维变量。粒子所处的位置为 $P_{position}=\left(\theta,v,t_{drop_1},t_{det_1},t_{drop_2},t_{det_2},t_{drop_3},t_{det_3}\right)$，将第 i 个粒子表示成 $x_i=\left(x_{i1},x_{i2},x_{i3},...,x_{i8}\right)$，其中每个 x_i 都可由目标函数求得函数值。粒子速度也是 8 维变量 $P_v=\left(\Delta\theta,\Delta v,\Delta t_{drop_1},\Delta t_{det_1},\Delta t_{drop_2},\Delta t_{det_2},\Delta t_{drop_3},\Delta t_{det_3}\right)$，将第 i 个粒子的速度表示为 $v_i=\left(v_{i1},v_{i2},v_{i3},...,v_{i8}\right)$。

对于第 $i+1$ 个粒子，其第 d 维的速度 v_{id}^{i+1} 变化公式为

$$v_{id}^{i+1}=w\cdot v_{id}^{i}+c_1\cdot rand()\cdot(pbest_{id}^{i}-x_{id}^{i})+c_2\cdot rand()\cdot(gbest_{d}^{i}-x_{id}^{i}) \quad (1.1)$$

其中 w 为惯性权重，默认为 0.5；c_1 为个体学习因子，c_2 为群体学习因子，均设为默认值 1.5；$rand()$ 为 $[0,1]$ 间的随机数。$pbest_i=\left(pbest_{i1},pbest_{i2},...,pbest_{i8}\right)$ 为每个粒子记忆的自己找到的最好位置，$gbest=\left(gbest_1,gbest_2,...,gbest_8\right)$ 为整个粒子群中最好的位置。

算法流程：

Step1 初始化粒子群

设粒子数量为 80，最大迭代次数为 100，从 GA 结果开始进行计算，随机产生粒子的速度。

Step2 评价粒子

计算每个粒子的适应度，即由目标函数得到函数值的负数（粒子代表的烟幕方案有效遮蔽时间的负数）。

Step3 更新最优

通过比较对应函数值的大小，更新个体最优 *pbest* 和全局最优 *gbest*。

Step4 更新粒子

按照式(1.1)更新粒子速度和位置。

Step5 停止条件

循环回到步骤 2，直到终止条件满足。终止条件为：

a) 最大迭代次数 100：迭代次数达到 100 代
b) 函数值容差 10^{-5}：适应度函数值变化小于 10^{-5} 时
c) 停滞代数 20 代：适应度在连续 20 代内没有显著改善

Ⅲ 模拟退火

模拟退火（Simulated annealing algorithm，SA）[2] [3]是一种来源于固体退火降温过程模拟的概率优化算法。它通过引入“温度”参数控制搜索过程，允许以一定概率接受较差解，以跳出局部最优。随着温度逐渐降低，算法趋于稳定，最终收敛到全局最优解附近。

本问中用该算法在 PSO 后期进行局部精细搜索。用 *rand*() 函数引入了随机性，允许暂时性的性能下降。速度向量定义了邻域范围，通过位置更新实现邻域搜索。此外，通过迭代次数的增加，逐渐减少搜索范围。

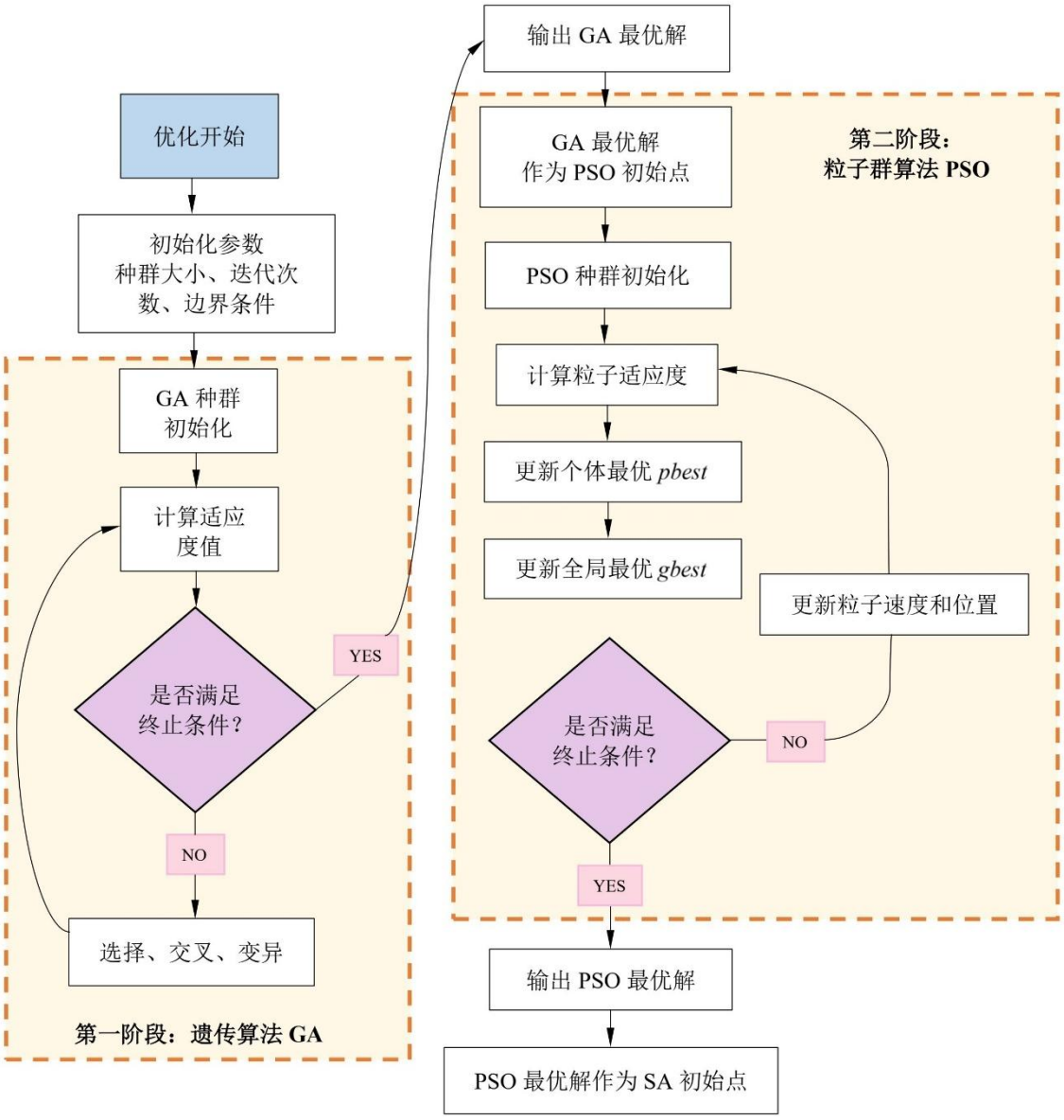


图 3-1 问题三算法流程图

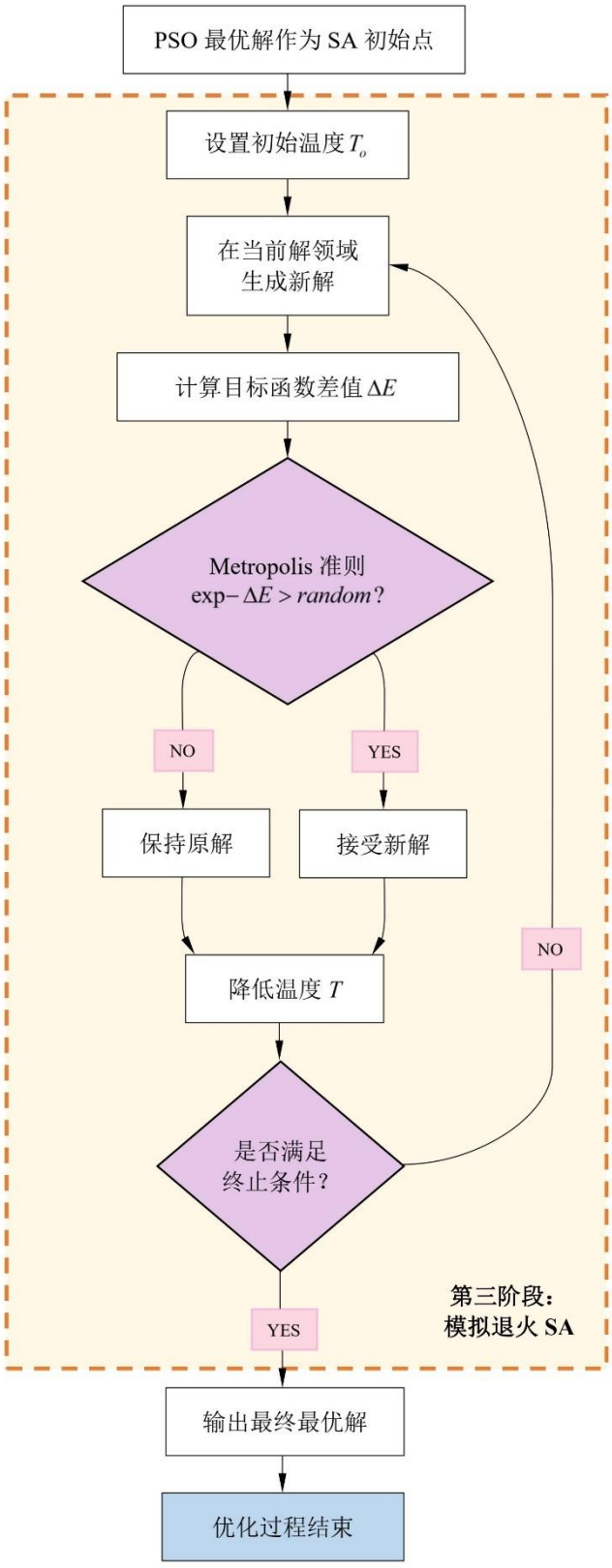


图 3-2 问题三算法流程图

4.3.4 问题三的求解结果

经过 MATLAB 代码优化计算，详细结果见 result1.xlsx，最大总有效遮蔽时长为：

6.3900s

投放顺序	投放延迟(s)	投放点	起爆延迟(s)	起爆点	有效遮蔽时长(s)	最大总遮蔽时长(s)
1	0.0000	(17800.0000, 0.0000, 1800.0000)	0.0543	(17805.5456, 0.9411, 1799.9856)	2.82	6.3900
2	1.0000	(17902.2031, 17.3434, 1800.0000)	0.0002	(17902.2230, 17.3468, 1800.0000)	3.65	
3	24.3128	(20284.8419, 421.6661, 1800.0000)	5.2664	(20823.0841, 513.0033, 1663.9601)	0	

表 1 问题三求解结果

屏蔽时间段为：

1.0043s~7.3843s



图 4 问题三有效屏蔽时间

4.4 问题四：三架无人机各投放一枚烟幕干扰弹对导弹 M1 的干扰

根据第四问所给数据，我们可以得到三架无人机 FY1、FY2、FY3,一颗导弹 M1，两者的空间位置已知，各投放的一枚烟幕弹的速度也已知。我们根据 FY1、FY2、FY3 和 M1 的位置坐标，确定烟幕干扰弹的投放策略，使得遮蔽时间尽可能长。具体分析如下：

4.4.1 问题分析

问题四在问题二、三的基础上，无人机数量变为多架。由于三架无人机都将投放烟幕弹，烟幕弹对导弹的遮蔽有效性的判定条件发生变化。需要建立多架无人机烟幕遮蔽有效性模型，并通过算法优化，确定烟幕弹投放策略，使得对导弹 M1 的有效遮蔽时间最大化。

4.4.2 多架无人机烟幕遮蔽有效性模型

Ⅰ单球遮蔽

根据 4.1.2，导弹位置 $\overrightarrow{M_i}$ 与真目标上任意一点 $\vec{Q}_i$ 构成线段 M_iP_i 上的任意点可表示为 $\overrightarrow{M_i}+s\times\left(\overrightarrow{P_i}-\overrightarrow{M_i}\right)$，其中 $s\in[0,1]$。

同一时刻可能存在多个烟幕球（干扰弹起爆后的云团），将 FYi 架无人机投出的第 j 枚烟幕干扰弹起爆后形成的烟幕球球心记为 C_{ij}。

该点在球心为 C_{ij} 的球面上时，到球心距离满足：$\left\|\overrightarrow{P_i}+s\cdot\left(\overrightarrow{P_i}-\overrightarrow{M_i}\right)-\overrightarrow{C_{ij}}\right\|=R$。其中 $R=10$。

展开后可转化为关于 s 的二次方程：

$$a\times s^2+b\times s+c=0$$

其中 $a=\left\|\overrightarrow{P_i}-\overrightarrow{M_i}\right\|^2$，$b=-2\times\left(\overrightarrow{C_{ij}}-\overrightarrow{M_i}\right)\times\left(\overrightarrow{P_i}-\overrightarrow{M_i}\right)$，$c=\left\|\overrightarrow{C_{ij}}-\overrightarrow{M_i}\right\|^2-R^2$。

关于 s 的二次方程若存在 $s\in[0,1]$ 的解，则线段 M_iP_i 上存在某点落在球心为 C_{ij} 的烟幕球面或烟幕球内。即对于判别式 $\Delta=b^2-4ac$，若 $\Delta\geq0$，则存在两个实根 s_1 和 s_2 $(s_1\leq s_2)$，若与此同时满足 $s_2\geq0$ 且 $s_1\leq1$，则线段与球心为 C_{ij} 的球相交，$\vec{Q}_i$ 被有效遮蔽。

Ⅱ遮蔽有效性判定

多枚烟幕干扰弹对真目标形成有效遮蔽的条件为：对于真目标上的任意一点，存在至少一枚烟幕弹阻挡了导弹和它之间的视线。烟幕干扰弹的延迟投放时间、延迟起爆时间不同，亦需考虑烟幕球的存在性。

MATLAB 代码通过生成高密度采样点，对每个时刻遍历所有采样点是否满足条件，只有所有点均满足条件时，认为烟幕球对导弹产生有效遮蔽。

采样点包括：底面和顶面圆周点、侧面点、内部网格点、轴线点。确保覆盖整个目标体积，避免漏判。

在每个时刻 t，首先判断三个烟幕球的存在性，即 $\mathrm{II}\left(t_{b_{ij}}\leq t\leq t_{b_{ij}}+20\right)=1$ 时存在，反之不存在。其中 $t_{b_{ij}}=t_{drop_{ij}}+t_{det_{ij}}$，$t_{drop_{ij}}$ 为 FYi 架无人机投出的第 j 枚烟幕干扰弹的延迟投放时间，$t_{det_{ij}}$ 为其延迟起爆时间。

对每一个采样点，判断 $\mathrm{II}\left(t_{b_{ij}}\leq t\leq t_{b_{ij}}+20\right)=1$ 的烟幕球 C_{ij} 能否有效遮蔽。若每个采样点均存在至少一个烟幕球能有效遮蔽，则该时刻 t 真目标被有效遮蔽。

4.4.3 遗传算法优化求解

Ⅰ 染色体编码

采用实数编码方式，每个染色体包含 12 个基因：

$$\text{染色体}=\left[\theta_1,v_1,t_{drop_1},t_{det_1},\theta_2,v_2,t_{drop_2},t_{det_2},\theta_3,v_3,t_{drop_3},t_{det_3}\right]$$

其中：$\theta_i\in[0,2\pi]$，为无人机 FYi 的飞行方向角；

$v_i\in[70,140]$，为无人机 FYi 的飞行速度；

$t_{drop_i}\in[0,70]$，无人机 FYi 烟幕弹延迟投放时间；

$t_{det_i}\in[0.1,50]$，无人机 FYi 烟幕弹延迟起爆时间。

Ⅱ 初始群体设计

采用混合初始化策略以提高种群多样性，20%的个体基于单机优化构造，即在第二问最优解附近生成变异个体，剩下 80%个体完全随机生成。

Ⅲ 适应度函数

适应度函数为总有效遮蔽时间的负值。

$$f_{fitness}(x) = -\sum_{t=0}^{Tend} \delta(t) \cdot \Delta t$$

其中 $\delta(t) = \text{II}(\text{时刻}t\text{真目标被有效遮蔽})$，为指示函数，$\Delta t$ 为时间步长。采用时间步进法，对每个时间步，运用多架无人机烟幕遮蔽有效性模型，计算烟幕弹对真目标是否实现有效遮蔽，得到指示函数的值。

Ⅳ 遗传操作设计

1. 选择操作

将锦标赛选择与精英保留相结合，设定锦标赛大小为 3，精英保留比例为 5%，从种群中随机选择若干个体，选择其中适应度最好的个体进入下一代。

2. 交叉操作

针对问题设计两种交叉算子：

整体交叉时，随机选择交叉点后，交换两个父代个体中对应无人机的全部四个参数；参数交叉则在不同无人机的同类参数之间进行基因交换。整体交叉的概率设置为 0.6，参数交叉的概率设置为 0.4。

执行交叉操作时，根据交叉概率随机选择参与交叉操作的父代个体，后根据随机数决定采用哪种交叉方式。对于整体交叉，随机选择无人机编号作为交叉点；对于参数交叉，随机选择参数类型作为交叉点。随后执行基因交换操作，生成新的子代个体。

3. 变异操作

采用自适应变异策略，变异概率随代数增加而减小，从初始的 0.2 线性降低到最终的 0.05。根据自适应变异概率决定是否对个体进行变异，然后随机选择变异类型和变异位置，最后按照相应的变异规则修改基因值。

Ⅴ 并行计算加速

为提高算法效率，采用了基于 MATLAB Parallel Computing Toolbox 的并行适应度评估，每代种群被平均分配到多个工作进程中进行并行评估，显著减少了计算时间。

4.4.4 问题四的求解结果

GA 算法存在随机性，故让算法独立运行多次，并适当增加了种群大小和最大代数，最后选择对应总遮蔽时间最长的解作为最终求解结果，详细结果见 result2.xlsx，最大总有效遮蔽时长为：

14.2s

无人机	无人机运动方向(度)	无人机运动速度(m/s)	投放点	起爆点	有效遮蔽时长(s)	最大总遮蔽时长(s)
1	6.22	134.39	(17807.37,0.80,1800)	(17902.75,11.20,1797.50)	4.8	14.2

2	212.64	132.30	(10909.53,701.67,1400)	(9899.93,55.13,997.62)	4.6	
3	136.80	114.46	(3447.37,-603.12,700)	(2669.76,127.05,274.42)	4.8	

表 2 问题四求解结果

4.5 问题五：五架无人机投放至多三枚烟幕干扰弹对三枚导弹的干扰

根据第五问所给数据，我们可以得到五架无人机 FY1、FY2、FY3、FY4、FY5,三颗导弹 M1、M2、M3，两者的空间位置已知，各投放的三枚烟幕弹的速度也已知。我们根据 FY1、FY2、FY3 和 M1、M2、M3 的位置坐标，确定烟幕干扰弹的投放策略，使得遮蔽时间尽可能长。具体分析如下：

4.5.1 问题分析

第五问在第四问的基础上，每个类型的决策变量都变成多个，即多架无人机、分别可投放的多枚烟幕干扰弹与需要实施对多枚来袭导弹的干扰。需要建立数学模型，并通过算法优化，确定烟幕弹投放策略，使得烟幕干扰弹对真目标的有效遮蔽时间尽可能长。

4.5.2 多架无人机烟幕遮蔽有效性模型在存在多枚导弹下的情况

遮蔽有效性的判断基于视线阻断原理。某一时刻 t，三枚导弹到真目标的“视线”被烟幕球同时阻挡，则该时刻烟幕球实施了有效干扰，若存在任意一条“视线”未被阻挡，则干扰无效。在 4.4.2 多架无人机烟幕遮蔽有效性模型的基础上，问题五对于多枚导弹（M1、M2、M3），需要对每枚导弹单独进行上述遮蔽判断，并确保所有导弹都被有效遮蔽。

4.5.3 模型建立

对于无人机 FYi 所投放的第 j 枚烟雾干扰弹 X_{ij}，决策变量包括：

$$\theta_i \in [0,2\pi], i=1,2,...,5$$
$$v_i \in [70,140], i=1,2,...,5$$
$$t_{drop_{ij}} \in [0,70], i=1,2,...,5$$
$$t_{det_{ij}} \in [0.1,50], i=1,2,...,5$$

目标函数：

$$\max \sum_{k=1}^{3} \sum_{m=1}^{M_k} \Delta t_{km}$$

其中 M_k 为干扰导弹 k 的烟幕弹数量，Δt_{km} 为第 m 枚烟幕弹对导弹 k 的有效遮蔽时间。

约束条件：

$$\begin{cases} \left| t_{drop_{ij}} - t_{drop_{ik}} \right| \geq 1s, \forall j \neq k \\ z_{det} \geq 0, t_b \leq T_{\max} = 20s \\ \sum_j x_{ij} \leq 3, \forall i \end{cases}$$

其中 x_{ij} 为指示变量，表示无人机 i 是否投放烟幕弹 j。

4.5.4 粒子群优化算法的求解过程

Ⅰ迭代优化过程

该算法采用多轮迭代的方式，逐步为每架无人机分配烟幕弹。在每轮迭代中，对于尚未达到烟幕弹数量上限的无人机，算法遍历所有导弹，使用优化算法求解针对该导弹的最优烟幕弹参数配置。

Ⅱ粒子群优化算法

问题三中运用粒子群算法，每一个粒子为一个完整的烟幕弹投放策略方案，进行局部精细搜索快速收敛到优质区域。此处，在每轮迭代中，使用粒子群优化算法优化单个烟幕弹的参数，每一个粒子为$\left(v,\theta,t_{drop},t_{det}\right)$，其所在位置可以表示为 $x_i=\left(x_{i1},x_{i2},x_{i3},x_{i4}\right)$。适应度函数 $f_{fitness}\left(x\right)=\Delta t\left(x\right)$，为烟幕弹的有效遮蔽时间。设置参数包括：种群大小为 50，最大迭代次数为 50，学习因子 1.5。通过更新迭代粒子的位置和速度，寻找最优解。

其中速度更新公式为：

$$v_{id}^{i+1}=w\cdot v_{id}^{i}+c_1\cdot rand()\cdot(pbest_{id}^{i}-x_{id}^{i})+c_2\cdot rand()\cdot(gbest_{d}^{i}-x_{id}^{i})$$

位置更新公式为：

$$x_{id}^{i+1}=x_{id}^{i}+v_{id}^{i+1}$$

其中 $pbest_i=\left(pbest_{i1},pbest_{i2},...,pbest_{i4}\right)$为每个粒子记忆的自己找到的最好位置，$gbest=\left(gbest_1,gbest_2,...,gbest_4\right)$为整个粒子群中最好的位置。

整体算法流程如下：

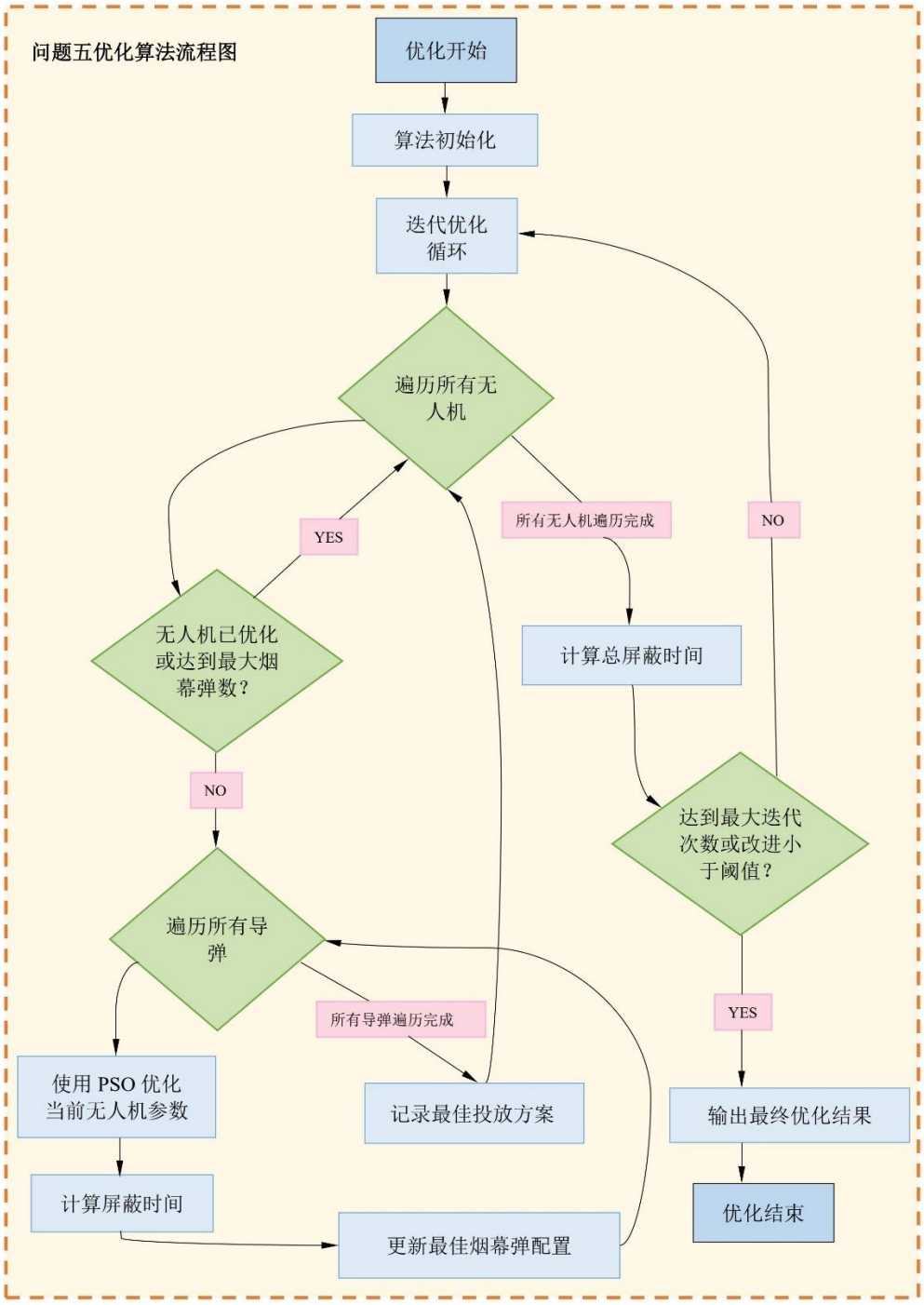


图 5 问题五优化算法流程图

4.5.5 问题五的求解结果

经过迭代，得到最终有效遮蔽时长最长的情况。详细结果见 result3.xlsx，最大总有效遮蔽时长为：

27.3s

烟幕弹编号	无人机编号	速度(m/s)	方向(°)	投放延迟(s)	起爆延迟(s)	干扰导弹	有效遮蔽时长(s)
S1	FY1	138.91	43.78	0	0.1	M1	4.1
S2	FY1	76.29	0	1.83	0.1	M1	3
S3	FY2	96.69	233.97	8.62	4.76	M2	0.3
S4	FY3	137.47	119.82	19.11	6.85	M1	2.2
S5	FY3	140	83.26	21.79	0.1	M1	0.8
S6	FY1	70	360	2.98	0.1	M1	1.7
S7	FY5	136.93	100.7	12.42	2.68	M1	3.2
S8	FY4	136.6	238.11	7.33	12.49	M3	1.3
S9	FY2	70.4	230.31	17.53	7.91	M1	2.9
S10	FY4	140	263.46	4.99	11.67	M3	0.7
S11	FY4	140	297.1	3.1	9.8	M2	3.6
S12	FY2	125.97	270.16	5.31	5.73	M1	3.5

表 3 问题五的求解结果

五、模型的检验与分析

5.1 采样点密度充分性分析

对于本题的每一问，都生成了覆盖整个真目标体积的采样点，在计算遮蔽时间时使用采样点的坐标判断遮蔽是否有效。采样点包括底面和顶面圆周点、侧面点、内部网格点、轴线点，分布在真目标的不同方位。

定义一组采样点数量

$$[1,5,10,50,100,200,500,1000,1500,2000,2581,3000,4000,5000],$$

运行问题一的代码生成基准采样点集合（2581 个点）。对于数量少于 2581 的采样点数量，从中随机抽取不同数量的点；对于数量多于 2581 的采样点数量，再生成足够多的采样点补充抽取，由此来模拟不同密度，计算屏蔽时间。随机抽取时，使用 *randperm* 函数来随机选择索引由于随机抽取，对每个数量都重复多次计算取平均值，以减少随机性的影响。最终所得的采样点数量对遮蔽时间准确性的影响如下图。

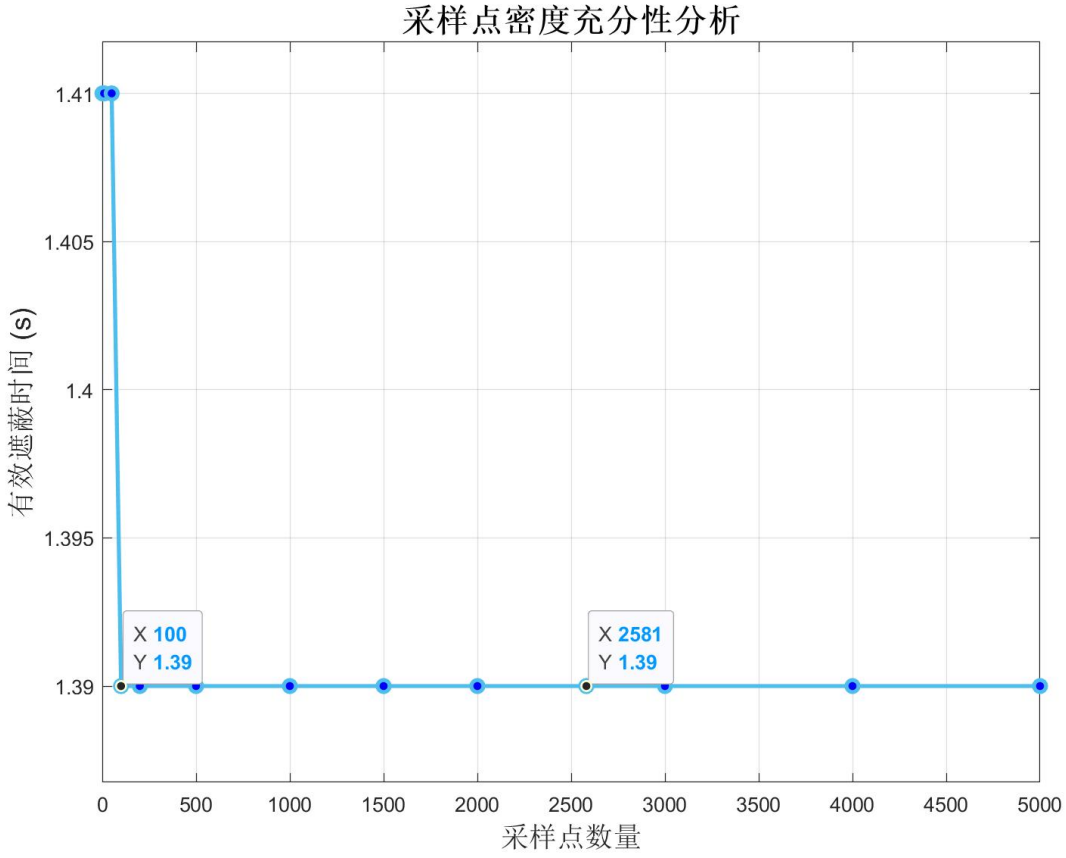


图 6 采样点密度充分性分析

根据曲线图，采样点数量达到 10 后遮蔽时间收敛到最终1% 以内，达到 100 后即趋于平缓，而模型中均取高于 500 个采样点，位于曲线平缓区域，且继续增加数量对结果影响很小，说明采样密度充分性较强。

5.2 遗传算法收敛性分析

整个模型的核心是一个优化问题，即最大化有效屏蔽时间。问题二开始，我们使用遗传算法进行广域搜索，为评定这种算法的稳定性，检验其收敛性能。

将遗传算法独立运行了 10 次，得到最大遮蔽时间为 4.65s，标准差为$\sigma = 0.1665$ s，最优结果为4.65s，变异系数为3.688%。故算法具有良好的稳定性和鲁棒性，始终能找到非常接近的解。同时，解空间具有良好结构，算法能可靠地找到高质量解。

从收敛曲线看，算法平均在 18 代后收敛到最优解的 99%范围内，显示出良好的搜索效率；所有运行的收敛曲线呈现相似的变化趋势，表明算法行为一致可靠；优化过程持续改进解质量，未出现过早收敛于局部最优的情况。

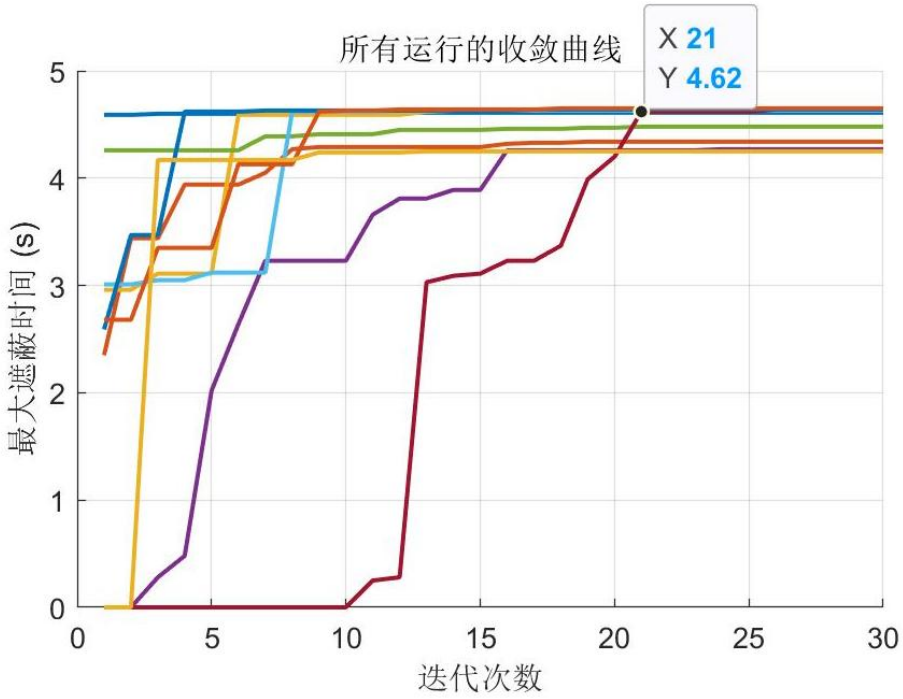


图 7 遗传算法收敛曲线

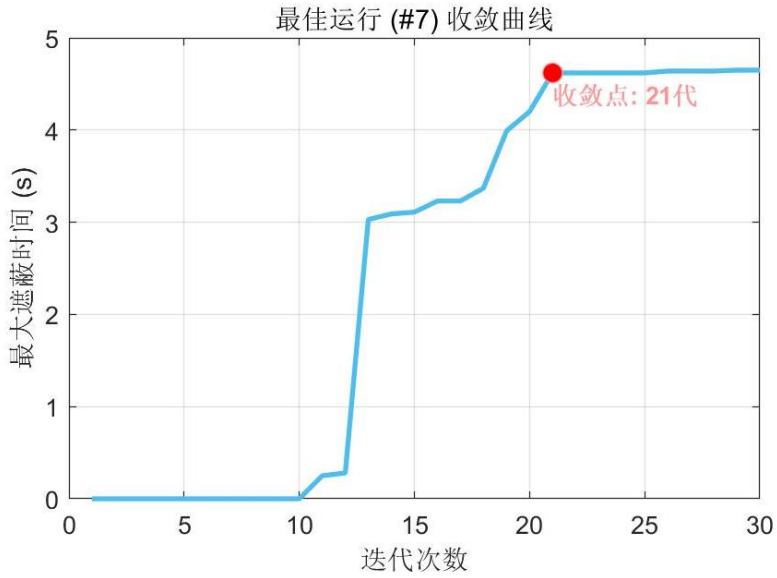


图 8 最佳运行收敛曲线

六、模型的评价

6.1 模型的优点

优点 1：建立的烟幕遮蔽有效性模型在一定程度上准确且巧妙描述了问题，且对其进行了相当的简化，易于理解操作。

优点 2：在烟幕干扰弹投放策略优化问题中，程序运行时间较快，空间占用很小。

优点 3：多维度变量考虑到烟幕干扰弹自身的影响因素。

优点 4：第四问和第五问在第一问的烟幕遮蔽有效性模型为基础构建，整体性强，架构清晰。

6.2 模型的缺点

缺点 1：由于采用了微分的思想进行了描述与求解，模型始终存在一定的误差。

参考文献

[1] 刘建华.粒子群算法的基本理论及其改进研究[D].中南大学,2009.

[2] 庞峰.模拟退火算法的原理及算法在优化问题上的应用[D].吉林大学,2006.

[3] DeepSeek, R1 0528, 深度求索（DeepSeek）, 2025-09-05

附录

问题一 MATLAB 代码实现

```
clear; clc;
g = 9.8;
eps = 1e-12;
dt = 0.01;
tgt_decoy = [0, 0, 0];
tgt_real.pos = [0, 200, 0];
tgt_real.r = 7;
tgt_real.h = 10;
uav.pos0 = [17800, 0, 1800];
uav.v = 120;
uav.t_drop = 1.5;
uav.t_det = 3.6;
smoke.r = 10;
smoke.v_sink = 3;
smoke.t_valid = 20;
missile.pos0 = [20000, 0, 2000];
missile.v = 300;
dir_xy = tgt_decoy(1:2) - uav.pos0(1:2);
dist_xy = norm(dir_xy);
if dist_xy < eps
    dir_xy = [0, 0];
else
    dir_xy = dir_xy / dist_xy;
end
dist_flight = uav.v * uav.t_drop;
pos_drop_xy = uav.pos0(1:2) + dist_flight * dir_xy;
pos_drop = [pos_drop_xy, uav.pos0(3)];
dist_tgt = norm(tgt_decoy(1:2) - pos_drop_xy);
if dist_tgt < eps
    dir_flight = [0, 0];
else
    dir_flight = (tgt_decoy(1:2) - pos_drop_xy) / dist_tgt;
end
pos_det_xy = pos_drop_xy + uav.v * uav.t_det * dir_flight;
drop_z = 0.5 * g * uav.t_det^2;
pos_det = [pos_det_xy, pos_drop(3) - drop_z];
fprintf('投放位置: (%.4f, %.4f, %.4f) m\n', pos_drop);
fprintf('起爆位置: (%.4f, %.4f, %.4f) m\n', pos_det);
n_circle = 60;
n_height = 20;
center_xy = tgt_real.pos(1:2);
z_min = tgt_real.pos(3);
z_max = tgt_real.pos(3) + tgt_real.h;
pts = [];
angles = linspace(0, 2*pi, n_circle);
for a = angles
    x = center_xy(1) + tgt_real.r * cos(a);
    y = center_xy(2) + tgt_real.r * sin(a);
    pts = [pts; [x, y, z_min]];
end
for a = angles
    x = center_xy(1) + tgt_real.r * cos(a);
    y = center_xy(2) + tgt_real.r * sin(a);
    pts = [pts; [x, y, z_max]];
end
```

```
    z_levels = linspace(z_min, z_max, n_height);
    for z = z_levels
        for a = angles
            x = center_xy(1) + tgt_real.r * cos(a);
            y = center_xy(2) + tgt_real.r * sin(a);
            pts = [pts; [x, y, z]];
        end
    end
    r_steps = linspace(0, tgt_real.r, 5);
    z_steps = linspace(z_min, z_max, 10);
    a_steps = linspace(0, 2*pi, 12);
    for z = z_steps
        for r = r_steps
            for a = a_steps
                x = center_xy(1) + r * cos(a);
                y = center_xy(2) + r * sin(a);
                pts = [pts; [x, y, z]];
            end
        end
    end
    pts = [pts; center_xy(1), center_xy(2), z_min];
    pts = [pts; center_xy(1), center_xy(2), z_min + tgt_real.h/4];
    pts = [pts; center_xy(1), center_xy(2), z_min + tgt_real.h/2];
    pts = [pts; center_xy(1), center_xy(2), z_min + 3*tgt_real.h/4];
    pts = [pts; center_xy(1), center_xy(2), z_max];
    pts = unique(pts, 'rows');
    n_pts = size(pts, 1);
    fprintf('采样点总数: %d\n', n_pts);
    dir_missile = tgt_decoy - missile.pos0;
    dist_missile = norm(dir_missile);
    if dist_missile < eps
        dir_m = [0, 0, 0];
    else
        dir_m = dir_missile / dist_missile;
    end
    t_det = uav.t_drop + uav.t_det;
    t_start = t_det;
    t_end = t_det + smoke.t_valid;
    t_list = t_start:dt:t_end;
    n_steps = length(t_list);
    t_total = 0;
    t_rec = [];
    shield_intv = [];
    prev_shield = false;
    for i = 1:n_steps
        t = t_list(i);
        pos_m = missile.pos0 + missile.v * t * dir_m;
        t_sink = t - t_det;
        pos_smoke = [pos_det(1), pos_det(2), pos_det(3) - smoke.v_sink * t_sink];
        all_covered = true;
        for j = 1:n_pts
            p = pts(j, :);
            v_p = p - pos_m;
            v_s = pos_smoke - pos_m;
            a = dot(v_p, v_p);
            if a < eps
                if norm(v_s) > smoke.r + eps
                    all_covered = false;
                    break;
```

```
                end
            else
                b = -2 * dot(v_p, v_s);
                c = dot(v_s, v_s) - smoke.r^2;
                disc = b^2 - 4*a*c;
                if disc < -eps
                    all_covered = false;
                    break;
                end
                if disc < 0
                    disc = 0;
                end
                sqrt_disc = sqrt(disc);
                s1 = (-b - sqrt_disc) / (2*a);
                s2 = (-b + sqrt_disc) / (2*a);

                if ~(s1 <= 1+eps && s2 >= -eps)
                    all_covered = false;
                    break;
                end
            end
        end
        if all_covered
            t_total = t_total + dt;
            t_rec = [t_rec; t, pos_m, pos_smoke];
            if ~prev_shield
                shield_intv = [shield_intv; t, nan];
            end
            prev_shield = true;
        else
            if prev_shield
                shield_intv(end, 2) = t - dt;
            end
            prev_shield = false;
        end
    end
    if prev_shield
        shield_intv(end, 2) = t_end;
    end
    fprintf('\n有效遮蔽总时长: %.4f 秒\n', t_total);
    if ~isempty(shield_intv)
        fprintf('遮蔽时间段:\n');
        for k = 1:size(shield_intv, 1)
            t1 = shield_intv(k, 1);
            t2 = shield_intv(k, 2);
            fprintf('段 %d: %.4f s 至 %.4f s, 持续: %.4f s\n', k, t1, t2, t2 -
t1);
        end
    else
        fprintf('无有效遮蔽时间段\n');
    end
```

问题二 MATLAB 代码实现

```
    function main()
    clc; close all;
    rng(42);
    a = 9.8;
```

```
    b = 1e-12;
    c = 0.01;
    d = [0, 0, 0];
    e.p = [0, 200, 0];
    e.r = 7;
    e.h = 10;
    f = [17800, 0, 1800];
    g = 10;
    h = 3;
    i = 20;
    j = [20000, 0, 2000];
    k = 300;
    l = -j / norm(j);
    fprintf('生成点...\n');
    m = gen_points(e);
    n = size(m, 1);
    fprintf('点数: %d\n', n);
    fprintf('开始计算...\n');
    o = [0, 70, 0, 0];
    p = [2*pi, 140, 60, 10];
    q = optimoptions('ga', ...
        'Display', 'iter', ...
        'MaxGenerations', 30, ...
        'PopulationSize', 40, ...
        'UseParallel', false, ...
        'OutputFcn', @track);
    global s;
    s.v = [];
    s.t = [];
    r = @(x) calc_time(x, f, j, k, l, g, h, i, a, b, c, e, m);
    [u, v, w, z] = ga(r, 4, [], [], [], [], o, p, [], q);
    fprintf('\n完成!\n');
    fprintf('结果:\n');
    fprintf('  角度: %.4f\n', u(1));
    fprintf('  速度: %.4f\n', u(2));
    fprintf('  投延: %.4f\n', u(3));
    fprintf('  爆延: %.4f\n', u(4));
    fprintf('时间: %.4f\n', -v);
    aa = u(1);
    bb = u(2);
    cc = u(3);
    dd = u(4);
    ee = [cos(aa), sin(aa), 0];
    ff = f + bb * cc * ee;
    gg = ff + bb * dd * ee - [0, 0, 0.5*a*dd^2];
    fprintf('投点: (%.4f, %.4f, %.4f)\n', ff);
    fprintf('爆点: (%.4f, %.4f, %.4f)\n', gg);
    hh = cc + dd;
    ii = hh;
    jj = hh + i;
    kk = ii:c:jj;
    ll = [];
    mm = false;
    nn = 0;
    oo = zeros(length(kk), 1);
    for pp = 1:length(kk)
        qq = kk(pp);
        rr = j + k * qq * l;
```

```
        ss = qq - hh;
        tt = [gg(1), gg(2), gg(3) - h * ss];
        uu = true;
        for vv = 1:min(100, n)
            ww = m(vv, :);
            xx = ww - rr;
            yy = tt - rr;
            zz = dot(xx, xx);
            if zz < b
                if norm(yy) > g + b
                    uu = false;
                    break;
                end
            else
                a1 = -2 * dot(xx, yy);
                a2 = dot(yy, yy) - g^2;
                a3 = a1^2 - 4*zz*a2;
                if a3 < -b
                    uu = false;
                    break;
                end
                if a3 < 0
                    a3 = 0;
                end
                a4 = sqrt(a3);
                a5 = (-a1 - a4) / (2*zz);
                a6 = (-a1 + a4) / (2*zz);
                if ~(a5 <= 1+b && a6 >= -b)
                    uu = false;
                    break;
                end
            end
        end
        oo(pp) = uu;
        if uu && ~mm
            mm = true;
            nn = qq;
        elseif ~uu && mm
            mm = false;
            ll = [ll; nn, qq-c];
        end
    end
    if mm
        ll = [ll; nn, jj];
    end
    fprintf('\n 时间段:\n');
    if isempty(ll)
        fprintf('  无\n');
    else
        for a7 = 1:size(ll, 1)
            a8 = ll(a7,2) - ll(a7,1);
            fprintf('  %d: %.4f - %.4f (%.4f)\n', a7, ll(a7,1), ll(a7,2), a8);
        end
    end
    fprintf('\n 画图...\n');
    figure('Position', [100, 100, 800, 600]);
    if ~isempty(s.v)
        plot(1:length(s.v), -s.v, 'LineWidth', 2);
        xlabel('次数');
```

```
        ylabel('时间');
        title('收敛');
        grid on;
    else
        if isfield(z, 'bestscore')
            plot(1:length(z.bestscore), -z.bestscore, 'LineWidth', 2);
            xlabel('次数');
            ylabel('时间');
            title('收敛');
            grid on;
        else
            text(0.5, 0.5, '无数据', 'HorizontalAlignment', 'center');
            axis off;
        end
    end
    figure('Position', [100, 100, 1200, 800]);
    subplot(2, 2, [1, 3]);
    a9 = max(kk) + 5;
    a10 = linspace(0, a9, 100);
    a11 = zeros(length(a10), 3);
    for a12 = 1:length(a10)
        a11(a12, :) = j + k * a10(a12) * l;
    end
    a13 = zeros(length(a10), 3);
    for a14 = 1:length(a10)
        a15 = a10(a14);
        if a15 >= hh
            a16 = a15 - hh;
            a13(a14, :) = [gg(1), gg(2), gg(3) - h * a16];
        else
            a13(a14, :) = [NaN, NaN, NaN];
        end
    end
    plot3(a11(:,1), a11(:,2), a11(:,3), 'r-', 'LineWidth', 2);
    hold on;
    a17 = ~isnan(a13(:,1));
    plot3(a13(a17,1), a13(a17,2), a13(a17,3), 'b--', 'LineWidth', 2);
    plot3(j(1), j(2), j(3), 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r');
    plot3(f(1), f(2), f(3), 'go', 'MarkerSize', 8, 'MarkerFaceColor', 'g');
    plot3(ff(1), ff(2), ff(3), 'mo', 'MarkerSize', 8, 'MarkerFaceColor', 'm');
    plot3(gg(1), gg(2), gg(3), 'co', 'MarkerSize', 8, 'MarkerFaceColor', 'c');
    plot3(d(1), d(2), d(3), 'ks', 'MarkerSize', 10, 'MarkerFaceColor', 'k');
    [a18, a19, a20] = cylinder(e.r, 20);
    a20 = a20 * e.h + e.p(3);
    a18 = a18 + e.p(1);
    a19 = a19 + e.p(2);
    surf(a18, a19, a20, 'FaceAlpha', 0.3, 'EdgeColor', 'none', 'FaceColor',
[0.8, 0.2, 0.2]);
    xlabel('X');
    ylabel('Y');
    zlabel('Z');
    title('轨迹');
    legend('导弹', '烟幕', '导弹起', '无人机起', '投点', '爆点', '假目标', '真目标
');
    grid on;
    axis equal;
    view(3);
    rotate3d on;
```

```
    subplot(2, 2, 2);
    plot(kk, oo, 'b-', 'LineWidth', 2);
    xlabel('时间');
    ylabel('状态');
    title('时间状态');
    ylim([-0.1, 1.1]);
    yticks([0, 1]);
    yticklabels({'否', '是'});
    grid on;
    subplot(2, 2, 4);
    a21 = {'角度', '速度', '投延', '爆延'};
    a22 = u;
    bar(a22);
    set(gca, 'XTickLabel', a21);
    ylabel('值');
    title('参数');
    grid on;
    figure('Position', [200, 200, 800, 600]);
    plot(a11(:,1), a11(:,2), 'r-', 'LineWidth', 2);
    hold on;
    plot(a13(a17,1), a13(a17,2), 'b--', 'LineWidth', 2);
    plot(j(1), j(2), 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r');
    plot(f(1), f(2), 'go', 'MarkerSize', 8, 'MarkerFaceColor', 'g');
    plot(ff(1), ff(2), 'mo', 'MarkerSize', 8, 'MarkerFaceColor', 'm');
    plot(gg(1), gg(2), 'co', 'MarkerSize', 8, 'MarkerFaceColor', 'c');
    plot(d(1), d(2), 'ks', 'MarkerSize', 10, 'MarkerFaceColor', 'k');
    a23 = linspace(0, 2*pi, 100);
    a24 = e.p(1) + e.r * cos(a23);
    a25 = e.p(2) + e.r * sin(a23);
    plot(a24, a25, 'k-', 'LineWidth', 2);
    xlabel('X');
    ylabel('Y');
    title('平面图');
    legend('导弹', '烟幕', '导弹起', '无人机起', '投点', '爆点', '假目标', '真目标
');
    grid on;
    axis equal;
    fprintf('完成!\n');
    end
    function points = gen_points(t)
        center = t.p(1:2);
        z1 = t.p(3);
        z2 = t.p(3) + t.h;
        num = 60;
        points = [];
        ang = linspace(0, 2*pi, num);
        for a = ang
            x = center(1) + t.r * cos(a);
            y = center(2) + t.r * sin(a);
            points = [points; [x, y, z1]];
        end
        for a = ang
            x = center(1) + t.r * cos(a);
            y = center(2) + t.r * sin(a);
            points = [points; [x, y, z2]];
        end
        zlev = linspace(z1, z2, 10);
        for z = zlev
```

```
            for a = ang(1:6:end)
                x = center(1) + t.r * cos(a);
                y = center(2) + t.r * sin(a);
                points = [points; [x, y, z]];
            end
        end
        points = [points; center(1), center(2), z1];
        points = [points; center(1), center(2), z2];

        points = unique(points, 'rows');
    end
    function time = calc_time(x, uav, mis, mis_v, mis_d, smk_r, snk_v, val_t,
grav, tol, dt, tgt, pts)
        ang = x(1);
        vel = x(2);
        drop_t = x(3);
        det_t = x(4);
        dir = [cos(ang), sin(ang), 0];
        drop_p = uav + vel * drop_t * dir;
        det_p = drop_p + vel * det_t * dir - [0, 0, 0.5*grav*det_t^2];
        if det_p(3) < 0
            time = 0;
            return;
        end
        burst = drop_t + det_t;
        start_t = burst;
        end_t = burst + val_t;
        t_arr = start_t:dt:end_t;
        total = 0;
        n_pts = size(pts, 1);
        idx = 1:min(50, n_pts);
        for i = 1:length(t_arr)
            t = t_arr(i);
            mis_p = mis + mis_v * t * mis_d;
            sink_t = t - burst;
            smk_c = [det_p(1), det_p(2), det_p(3) - snk_v * sink_t];
            all_ok = true;
            for j = idx
                p = pts(j, :);
                v1 = p - mis_p;
                v2 = smk_c - mis_p;
                a = dot(v1, v1);
                if a < tol
                    if norm(v2) > smk_r + tol
                        all_ok = false;
                        break;
                    end
                else
                    b = -2 * dot(v1, v2);
                    c_val = dot(v2, v2) - smk_r^2;
                    disc = b^2 - 4*a*c_val;
                    if disc < -tol
                        all_ok = false;
                        break;
                    end
                    if disc < 0
                        disc = 0;
                    end
                    sqrt_d = sqrt(disc);
```

```
                s1 = (-b - sqrt_d) / (2*a);
                s2 = (-b + sqrt_d) / (2*a);
                if ~(s1 <= 1+tol && s2 >= -tol)
                    all_ok = false;
                    break;
                end
            end
        end
        if all_ok
            total = total + dt;
        end
    end
    time = -total;
end
function [state, options, optchanged] = track(options, state, flag)
    global s;
    switch flag
        case 'init'
            s.v = [];
            s.t = [];
        case 'iter'
            best = min(state.Score);
            s.v = [s.v; best];
            s.t = [s.t; state.Generation];
        case 'done'
    end
    optchanged = false;
end
```

问题三 MATLAB 代码实现

```
clear; clc; close all;
rng(42, 'twister');
gravity = 9.81;
tolerance = 1e-10;
time_step = 0.01;
decoy_target = [0, 0, 0];
real_target.center = [0, 200, 0];
real_target.radius = 7;
real_target.height = 10;
UAV.initial_position = [17800, 0, 1800];
smoke_cloud.effective_radius = 10;
smoke_cloud.settling_velocity = 3;
smoke_cloud.effective_duration = 20;
missile.initial_position = [20000, 0, 2000];
missile.velocity = 300;
direction_vector = -missile.initial_position;
missile.direction = direction_vector / norm(direction_vector);
fprintf('正在生成目标表面采样点...\n');
sampling_points = generate_sampling_points(real_target);
point_count = size(sampling_points, 1);
fprintf('成功生成采样点数量: %d\n', point_count);
fprintf('开始多算法优化计算...\n');
lower_bounds = [0, 70, 0, 0, 1, 0, 2, 0];
upper_bounds = [2*pi, 140, 10, 10, 20, 10, 30, 10];
ga_options = optimoptions('ga', ...
    'Display', 'iter', ...
    'MaxGenerations', 50, ...
```

```
        'PopulationSize', 100, ...
        'UseParallel', true, ...
        'FunctionTolerance', 1e-4);
    objective_function = @(x) calculate_total_coverage(x,
UAV.initial_position, ...
        missile, smoke_cloud, gravity, tolerance, time_step, real_target,
sampling_points);
    fprintf('执行遗传算法优化...\n');
    [ga_opt_params, ga_opt_value] = ga(objective_function, 8, [], [], [],
[], ...
        lower_bounds, upper_bounds, [], ga_options);
    fprintf('\n遗传算法优化完成!\n');
    fprintf('遗传算法最优遮蔽时间: %.4f 秒\n', -ga_opt_value);
    pso_options = optimoptions('particleswarm', ...
        'Display', 'iter', ...
        'MaxIterations', 100, ...
        'SwarmSize', 80, ...
        'UseParallel', true, ...
        'InitialSwarmMatrix', ga_opt_params, ...
        'FunctionTolerance', 1e-5, ...
        'HybridFcn', @fmincon);
    fprintf('执行粒子群算法优化...\n');
    [pso_opt_params, pso_opt_value] = particleswarm(objective_function, 8, ...
        lower_bounds, upper_bounds, pso_options);
    fprintf('\n粒子群算法优化完成!\n');
    fprintf('粒子群算法最优遮蔽时间: %.4f 秒\n', -pso_opt_value);
    sa_options = optimoptions('simulannealbnd', ...
        'Display', 'iter', ...
        'MaxIterations', 500, ...
        'TemperatureFcn', @temperaturefast, ...
        'AnnealingFcn', @annealingfast, ...
        'ReannealInterval', 100, ...
        'FunctionTolerance', 1e-6);
    if -pso_opt_value > -ga_opt_value
        initial_point = pso_opt_params;
        fprintf('采用粒子群算法的结果作为模拟退火初始点\n');
    else
        initial_point = ga_opt_params;
        fprintf('采用遗传算法的结果作为模拟退火初始点\n');
    end
    fprintf('执行模拟退火算法优化...\n');
    [sa_opt_params, sa_opt_value] = simulannealbnd(objective_function, ...
        initial_point, lower_bounds, upper_bounds, sa_options);
    fprintf('\n模拟退火算法优化完成!\n');
    fprintf('模拟退火算法最优遮蔽时间: %.4f 秒\n', -sa_opt_value);
    algorithm_results = [
        -ga_opt_value, ga_opt_params;
        -pso_opt_value, pso_opt_params;
        -sa_opt_value, sa_opt_params
    ];
    [best_coverage_time, best_algorithm_index] = max(algorithm_results(:, 1));
    optimal_parameters = algorithm_results(best_algorithm_index, 2:end);
    final_opt_value = -best_coverage_time;
    algorithm_names = {'遗传算法', '粒子群算法', '模拟退火算法'};
    fprintf('\n%s 获得最佳优化结果: %.4f 秒\n', ...
        algorithm_names{best_algorithm_index}, best_coverage_time);
    flight_direction = optimal_parameters(1);
    flight_speed = optimal_parameters(2);
```

```
    drop_delay = [optimal_parameters(3), optimal_parameters(5),
optimal_parameters(7)];
    detonation_delay = [optimal_parameters(4), optimal_parameters(6),
optimal_parameters(8)];
    direction_unit_vector = [cos(flight_direction), sin(flight_direction), 0];
    drop_points = zeros(3, 3);
    detonation_points = zeros(3, 3);
    detonation_times = zeros(1, 3);
    for i = 1:3
        drop_points(i, :) = UAV.initial_position + flight_speed * drop_delay(i) *
direction_unit_vector;
        detonation_points(i, :) = drop_points(i, :) + flight_speed *
detonation_delay(i) * ...
            direction_unit_vector - [0, 0, 0.5*gravity*detonation_delay(i)^2];
        detonation_times(i) = drop_delay(i) + detonation_delay(i);
    end
    fprintf('\n最优参数组合:\n');
    fprintf('飞行方向: %.2f°\n', rad2deg(flight_direction));
    fprintf('飞行速度: %.2f m/s\n', flight_speed);
    for i = 1:3
        fprintf('弹%d 投放延迟: %.2f s, 起爆延迟: %.2f s\n', i, drop_delay(i),
detonation_delay(i));
    end
    fprintf('\n投放点和起爆点坐标:\n');
    for i = 1:3
        fprintf('弹%d 投放点: (%.4f, %.4f, %.4f) m\n', i, drop_points(i, :));
        fprintf('弹%d 起爆点: (%.4f, %.4f, %.4f) m\n', i,
detonation_points(i, :));
    end
    start_time = min(detonation_times);
    end_time = max(detonation_times) + smoke_cloud.effective_duration;
    time_points = start_time:time_step:end_time;
    coverage_intervals = [];
    is_covered = false;
    interval_start = 0;
    coverage_status = zeros(length(time_points), 1);
    for idx = 1:length(time_points)
        current_time = time_points(idx);
        missile_position = missile.initial_position + missile.velocity *
current_time * missile.direction;
        covered = false;
        if current_time >= detonation_times(1) && current_time <=
detonation_times(1) + smoke_cloud.effective_duration
            settling_time = current_time - detonation_times(1);
            smoke_center = [detonation_points(1, 1), detonation_points(1, 2), ...
                detonation_points(1, 3) - smoke_cloud.settling_velocity *
settling_time];
            if check_coverage_status(missile_position, smoke_center, ...
                    smoke_cloud.effective_radius, sampling_points, tolerance)
                covered = true;
            end
        end
        if ~covered && current_time >= detonation_times(2) && ...
                current_time <= detonation_times(2) +
smoke_cloud.effective_duration
            settling_time = current_time - detonation_times(2);
            smoke_center = [detonation_points(2, 1), detonation_points(2, 2), ...
```

```
                detonation_points(2, 3) - smoke_cloud.settling_velocity *
settling_time];
            if check_coverage_status(missile_position, smoke_center, ...
                    smoke_cloud.effective_radius, sampling_points, tolerance)
                covered = true;
            end
        end
        if ~covered && current_time >= detonation_times(3) && ...
                current_time <= detonation_times(3) +
smoke_cloud.effective_duration
            settling_time = current_time - detonation_times(3);
            smoke_center = [detonation_points(3, 1), detonation_points(3, 2), ...
                detonation_points(3, 3) - smoke_cloud.settling_velocity *
settling_time];
            if check_coverage_status(missile_position, smoke_center, ...
                    smoke_cloud.effective_radius, sampling_points, tolerance)
                covered = true;
            end
        end
        coverage_status(idx) = covered;
        if covered && ~is_covered
            is_covered = true;
            interval_start = current_time;
        elseif ~covered && is_covered
            is_covered = false;
            coverage_intervals = [coverage_intervals; interval_start,
current_time-time_step];
        end
    end
    if is_covered
        coverage_intervals = [coverage_intervals; interval_start, end_time];
    end
    fprintf('\n 遮蔽时间段详情:\n');
    if isempty(coverage_intervals)
        fprintf('  未检测到有效遮蔽时间段\n');
    else
        total_coverage_time = 0;
        for i = 1:size(coverage_intervals, 1)
            duration = coverage_intervals(i,2) - coverage_intervals(i,1);
            total_coverage_time = total_coverage_time + duration;
            fprintf('  时间段 %d: %.4f s - %.4f s (持续 %.4f s)\n', ...
                i, coverage_intervals(i,1), coverage_intervals(i,2), duration);
        end
        fprintf('总有效遮蔽时间: %.4f s\n', total_coverage_time);
    end
    fprintf('\n 保存优化结果到文件...\n')
    direction_degrees = rad2deg(flight_direction);
    if direction_degrees < 0
        direction_degrees = direction_degrees + 360;
    end
    result_matrix = cell(3, 10);
    for i = 1:3
        result_matrix{i, 1} = direction_degrees;
        result_matrix{i, 2} = flight_speed;
        result_matrix{i, 3} = i;
        result_matrix{i, 4} = drop_points(i, 1);
        result_matrix{i, 5} = drop_points(i, 2);
        result_matrix{i, 6} = drop_points(i, 3);
        result_matrix{i, 7} = detonation_points(i, 1);
```

```
        result_matrix{i, 8} = detonation_points(i, 2);
        result_matrix{i, 9} = detonation_points(i, 3);
        single_coverage_time = calculate_single_coverage(...
            flight_direction, flight_speed, drop_delay(i),
detonation_delay(i), ...
            drop_points(i, :), detonation_points(i, :), UAV.initial_position, ...
            missile, smoke_cloud, gravity, tolerance, time_step, real_target,
sampling_points);
        result_matrix{i, 10} = single_coverage_time;
    end
    output_filename = 'result1.xlsx';
    header_labels = {'无人机运动方向', '无人机运动速度 (m/s)', '烟幕干扰弹编号', ...
                     '烟幕干扰弹投放点的 x 坐标 (m)', '烟幕干扰弹投放点的 y 坐标
(m)', ...
                     '烟幕干扰弹投放点的 z 坐标 (m)', '烟幕干扰弹起爆点的 x 坐标
(m)', ...
                     '烟幕干扰弹起爆点的 y 坐标 (m)', '烟幕干扰弹起爆点的 z 坐标
(m)', ...
                     '有效干扰时长 (s)'};
    writecell(header_labels, output_filename, 'Sheet', 1, 'Range', 'A1');
    writecell(result_matrix, output_filename, 'Sheet', 1, 'Range', 'A2');
    note_text = {'注：以 x 轴为正向，逆时针方向为正，取值 0~360（度）。'};
    writecell(note_text, output_filename, 'Sheet', 1, 'Range', 'A6');
    fprintf('结果已保存至文件: %s\n', output_filename);
    fprintf('\n 生成可视化结果图表...\n');
    generate_visualizations(UAV.initial_position, missile, smoke_cloud, ...
        real_target, decoy_target, drop_points, detonation_points, ...
        detonation_times, coverage_intervals, flight_direction, flight_speed, ...
        drop_delay, detonation_delay, -ga_opt_value, -pso_opt_value, -
sa_opt_value);
    fprintf('计算程序执行完成!\n');
    function points = generate_sampling_points(target)
        center_xy = target.center(1:2);
        z_min = target.center(3);
        z_max = target.center(3) + target.height;
        points = [];
        angle_count = 60;
        angles = linspace(0, 2*pi, angle_count);
        for a = angles
            x = center_xy(1) + target.radius * cos(a);
            y = center_xy(2) + target.radius * sin(a);
            points = [points; [x, y, z_min]];
        end
        for a = angles
            x = center_xy(1) + target.radius * cos(a);
            y = center_xy(2) + target.radius * sin(a);
            points = [points; [x, y, z_max]];
        end
        z_levels = linspace(z_min, z_max, 10);
        for z = z_levels
            for a = angles(1:6:end)
                x = center_xy(1) + target.radius * cos(a);
                y = center_xy(2) + target.radius * sin(a);
                points = [points; [x, y, z]];
            end
        end
        points = [points; center_xy(1), center_xy(2), z_min];
        points = [points; center_xy(1), center_xy(2), z_max];
```

```
        points = unique(points, 'rows');
    end
    function total_time = calculate_total_coverage(params, uav_pos, missile,
smoke, g, eps, dt, target, samples)
        theta = params(1);
        v = params(2);
        t_drop1 = params(3);
        t_det1 = params(4);
        t_drop2 = params(5);
        t_det2 = params(6);
        t_drop3 = params(7);
        t_det3 = params(8);
        if t_drop2 < t_drop1 + 1 || t_drop3 < t_drop2 + 1
            total_time = 0;
            return;
        end
        u = [cos(theta), sin(theta), 0];
        P_drop1 = uav_pos + v * t_drop1 * u;
        P_det1 = P_drop1 + v * t_det1 * u - [0, 0, 0.5*g*t_det1^2];
        P_drop2 = uav_pos + v * t_drop2 * u;
        P_det2 = P_drop2 + v * t_det2 * u - [0, 0, 0.5*g*t_det2^2];
        P_drop3 = uav_pos + v * t_drop3 * u;
        P_det3 = P_drop3 + v * t_det3 * u - [0, 0, 0.5*g*t_det3^2];
        if P_det1(3) < 0 || P_det2(3) < 0 || P_det3(3) < 0
            total_time = 0;
            return;
        end
        t_burst1 = t_drop1 + t_det1;
        t_burst2 = t_drop2 + t_det2;
        t_burst3 = t_drop3 + t_det3;
        t_start = min([t_burst1, t_burst2, t_burst3]);
        t_end = max([t_burst1, t_burst2, t_burst3]) + smoke.effective_duration;
        t_list = t_start:dt:t_end;
        coverage_time = 0;
        sample_indices = 1:min(50, size(samples, 1));
        for i = 1:length(t_list)
            t = t_list(i);
            missile_pos = missile.initial_position + missile.velocity * t *
missile.direction;
            is_covered = false;
            if t >= t_burst1 && t <= t_burst1 + smoke.effective_duration
                sink_time = t - t_burst1;
                smoke_center = [P_det1(1), P_det1(2), P_det1(3) -
smoke.settling_velocity * sink_time];
                if check_coverage_status(missile_pos, smoke_center,
smoke.effective_radius, ...
                        samples(sample_indices, :), eps)
                    is_covered = true;
                end
            end
            if ~is_covered && t >= t_burst2 && t <= t_burst2 +
smoke.effective_duration
                sink_time = t - t_burst2;
                smoke_center = [P_det2(1), P_det2(2), P_det2(3) -
smoke.settling_velocity * sink_time];
                if check_coverage_status(missile_pos, smoke_center,
smoke.effective_radius, ...
```

```
                    samples(sample_indices, :), eps)
                is_covered = true;
            end
        end
        if ~is_covered && t >= t_burst3 && t <= t_burst3 +
smoke.effective_duration
            sink_time = t - t_burst3;
            smoke_center = [P_det3(1), P_det3(2), P_det3(3) -
smoke.settling_velocity * sink_time];
            if check_coverage_status(missile_pos, smoke_center,
smoke.effective_radius, ...
                    samples(sample_indices, :), eps)
                is_covered = true;
            end
        end
        if is_covered
            coverage_time = coverage_time + dt;
        end
    end
    total_time = -coverage_time;
end
function is_covered = check_coverage_status(missile_pos, smoke_center,
smoke_r, samples, eps)
    is_covered = true;
    for j = 1:size(samples, 1)
        p = samples(j, :);
        MP = p - missile_pos;
        MC = smoke_center - missile_pos;
        a = dot(MP, MP);
        if a < eps
            if norm(MC) > smoke_r + eps
                is_covered = false;
                return;
            end
        else
            b = -2 * dot(MP, MC);
            c = dot(MC, MC) - smoke_r^2;
            disc = b^2 - 4*a*c;
            if disc < -eps
                is_covered = false;
                return;
            end
            if disc < 0
                disc = 0;
            end
            sqrt_disc = sqrt(disc);
            s1 = (-b - sqrt_disc) / (2*a);
            s2 = (-b + sqrt_disc) / (2*a);

            if ~(s1 <= 1+eps && s2 >= -eps)
                is_covered = false;
                return;
            end
        end
    end
end
function coverage_time = calculate_single_coverage(theta, v, t_drop, t_det,
P_drop, P_det, ...
    uav_pos, missile, smoke, g, eps, dt, target, samples)
```

```
        t_burst = t_drop + t_det;
        t_start = t_burst;
        t_end = t_burst + smoke.effective_duration;
        t_list = t_start:dt:t_end;
        total_time = 0;
        sample_indices = 1:min(50, size(samples, 1));
        for i = 1:length(t_list)
            t = t_list(i);
            missile_pos = missile.initial_position + missile.velocity * t *
missile.direction;
            sink_time = t - t_burst;
            smoke_center = [P_det(1), P_det(2), P_det(3) -
smoke.settling_velocity * sink_time];
            if check_coverage_status(missile_pos, smoke_center,
smoke.effective_radius, ...
                    samples(sample_indices, :), eps)
                total_time = total_time + dt;
            end
        end
        coverage_time = total_time;
    end
    function generate_visualizations(uav_pos, missile, smoke, target, decoy, ...
        drop_points, detonation_points, detonation_times, coverage_intervals, ...
        theta, v, t_drop, t_det, ga_time, pso_time, sa_time)
        figure('Position', [100, 100, 1400, 1000], 'Name', '烟幕干扰弹投放策略分析
结果');
        subplot(3, 3, [1, 2, 4, 5]);
        hold on;
        grid on;
        axis equal;
       t_max = max(detonation_times) + smoke.effective_duration + 5;
        t_traj = linspace(0, t_max, 100);
        missile_traj = zeros(length(t_traj), 3);
        for i = 1:length(t_traj)
            missile_traj(i, :) = missile.initial_position + missile.velocity *
t_traj(i) * missile.direction;
        end
        plot3(missile_traj(:,1), missile_traj(:,2), missile_traj(:,3), 'r-',
'LineWidth', 2);
        plot3(uav_pos(1), uav_pos(2), uav_pos(3), 'go', 'MarkerSize', 10,
'MarkerFaceColor', 'g');
        colors = ['b', 'm', 'c'];
        for i = 1:3
            plot3(drop_points(i, 1), drop_points(i, 2), drop_points(i, 3), ...
                [colors(i) 'o'], 'MarkerSize', 8, 'MarkerFaceColor', colors(i));
            plot3(detonation_points(i, 1), detonation_points(i, 2),
detonation_points(i, 3), ...
                [colors(i) 's'], 'MarkerSize', 8, 'MarkerFaceColor', colors(i));
        end
        plot3(decoy(1), decoy(2), decoy(3), 'ks', 'MarkerSize', 10,
'MarkerFaceColor', 'k');
        [x_cyl, y_cyl, z_cyl] = cylinder(target.radius, 20);
        z_cyl = z_cyl * target.height + target.center(3);
        x_cyl = x_cyl + target.center(1);
        y_cyl = y_cyl + target.center(2);
        surf(x_cyl, y_cyl, z_cyl, 'FaceAlpha', 0.3, 'EdgeColor', 'none',
'FaceColor', [0.8, 0.2, 0.2]);
        xlabel('X (m)');
        ylabel('Y (m)');
```

```
        zlabel('Z (m)');
        title('导弹轨迹与烟幕弹空间分布');
        legend('导弹轨迹', '无人机初始位置', '弹 1 投放点', '弹 1 起爆点', ...
            '弹 2 投放点', '弹 2 起爆点', '弹 3 投放点', '弹 3 起爆点', '假目标', '真目标
');
        view(3);
        rotate3d on;
        subplot(3, 3, 3);
        hold on;
        grid on;
        for i = 1:size(coverage_intervals, 1)
            plot([coverage_intervals(i,1), coverage_intervals(i,2)], [1, 1], 'b-
', 'LineWidth', 10);
        end
        plot(detonation_times, [0.9, 0.9, 0.9], 'rv', 'MarkerSize', 10,
'MarkerFaceColor', 'r');
        xlabel('时间 (s)');
        ylabel('遮蔽状态');
        title('遮蔽时间段分布');
        ylim([0.8, 1.2]);
        set(gca, 'YTick', []);
        subplot(3, 3, 6);
        algorithms = {'遗传算法', '粒子群算法', '模拟退火算法'};
        times = [ga_time, pso_time, sa_time];
        bar(times, 'FaceColor', [0.5, 0.7, 0.9]);
        set(gca, 'XTickLabel', algorithms);
        ylabel('遮蔽时间 (s)');
        title('算法性能比较');
        grid on;
        for i = 1:length(times)
            text(i, times(i)+0.1, sprintf('%.2f s', times(i)), ...
                'HorizontalAlignment', 'center', 'FontWeight', 'bold');
        end
        subplot(3, 3, [7, 8, 9]);
        axis off;
        text(0.05, 0.9, '最优参数结果:', 'FontSize', 12, 'FontWeight', 'bold');
        text(0.05, 0.85, sprintf('飞行方向: %.2f°', rad2deg(theta)));
        text(0.05, 0.80, sprintf('飞行速度: %.2f m/s', v));
        text(0.05, 0.75, sprintf('弹 1: 投放%.1fs, 起爆%.1fs', t_drop(1),
t_det(1)));
        text(0.05, 0.70, sprintf('弹 2: 投放%.1fs, 起爆%.1fs', t_drop(2),
t_det(2)));
        text(0.05, 0.65, sprintf('弹 3: 投放%.1fs, 起爆%.1fs', t_drop(3),
t_det(3)));
        total_time = 0;
        if ~isempty(coverage_intervals)
            for i = 1:size(coverage_intervals, 1)
                total_time = total_time + (coverage_intervals(i,2) -
coverage_intervals(i,1));
            end
        end
        text(0.05, 0.55, sprintf('总遮蔽时间: %.2f s', total_time), 'FontSize',
12, 'FontWeight', 'bold');
        [best_time, best_idx] = max([ga_time, pso_time, sa_time]);
        text(0.05, 0.45, sprintf('最佳算法: %s (%.2f s)', algorithms{best_idx},
best_time), ...
            'FontSize', 12, 'FontWeight', 'bold', 'Color', [0, 0.5, 0]);
```

```
        if ~isempty(coverage_intervals)
            text(0.05, 0.35, '遮蔽时间段详情:', 'FontSize', 10, 'FontWeight',
'bold');
            y_pos = 0.30;
            for i = 1:min(3, size(coverage_intervals, 1))
                text(0.05, y_pos, sprintf('  段%d: %.1f-%.1fs (%.1fs)', i, ...
                    coverage_intervals(i,1), coverage_intervals(i,2), ...
                    coverage_intervals(i,2) - coverage_intervals(i,1)));
                y_pos = y_pos - 0.05;
            end
            if size(coverage_intervals, 1) > 3
                text(0.05, y_pos, sprintf('  还有%d 个时间段...',
size(coverage_intervals, 1)-3));
            end
        end
    end
```

问题四 MATLAB 代码实现

```
    clear; clc; close all;
    G = 9.8;
    smoke_r = 10+0*52241;
    sink_v = 3;
    valid_tIme = 20;
    dt1 = 0.05;
    t_end = 70;
    missile_pos0 = [20000, 0, 2000];
    missile_diR = [-20000, 0, -2000];
    missile_speed = 300;
    missile_diR = missile_diR / norm(missile_diR) * missile_speed;
    uav_poso = [
        17800, 0, 1800;
        12000, 1400, 1400;
        6000, -3000, 700
    ];
    target_Center = [0, 200, 0];
    target_radIus = 7;
    target_height = 10;
    target_points = gen_target_points(target_Center, target_radIus,
target_height, 30, 5);
    lb = [0, 70, 0, 0.1, 0, 70, 0, 0.1, 0, 70, 0, 0.1];
    ub = [2*pi, 140, 100, 50, 2*pi, 140, 100, 50, 2*pi, 140, 100, 50];
    options = optimoptions('ga', ...
        'Display', 'iter', ...
        'MaxGenerations', 1000, ...
        'PopulationSize', 10500, ...
        'UseParallel', true, ...
        'FunctionTolerance', 1e-2, ...
        'PlotFcn', {@gaplotbestf, @gaplotstopping});
    fprintf('开始遗传算法优化...\n');
    tic;
    [x_opt, fval, exitflag, output] = ga(@(x) objective_function(x, uav_poso,
missile_pos0, missile_diR, target_points, dt1, t_end, G, smoke_r, sink_v,
valid_tIme), ...
        12, [], [], [], [], lb, ub, [], options);
    optimization_time = toc;
    fprintf('优化完成，耗时: %.2f 秒\n', optimization_time);
```

```
    [drop_points, det_points, solo_times] = compute_points_and_times(x_opt,
uav_poso, missile_pos0, missile_diR, target_points, dt1, t_end, G, smoke_r,
sink_v, valid_tIme);
    write_to_excel(x_opt, drop_points, det_points, solo_times, 'result2.xlsx');
    fprintf('最优总遮蔽时间: %.2f 秒\n', -fval);
    fprintf('各无人机单独遮蔽时间: %.2f, %.2f, %.2f 秒\n', solo_times(1),
solo_times(2), solo_times(3));
    plot_optimization_results(x_opt, uav_poso, missile_pos0, missile_diR,
target_points, dt1, t_end, G, smoke_r, sink_v, valid_tIme);
    function point = gen_target_points(center, radius, height, n_angle,
n_height)
        point = [];
        for h = [0, height]
            for i = 1:n_angle
                angle = 2 * pi * (i-1) / n_angle;
                x = center(1) + radius * cos(angle);
                y = center(2) + radius * sin(angle);
                z = center(3) + h;
                point = [point; x, y, z];
            end
        end
        for i = 1:n_height
            z = center(3) + height * (i-1) / (n_height-1);
            for j = 1:n_angle
                angle = 2 * pi * (j-1) / n_angle;
                x = center(1) + radius * cos(angle);
                y = center(2) + radius * sin(angle);
                point = [point; x, y, z];
            end
        end
        point = [point; center(1), center(2), center(3)];
        point = [point; center(1), center(2), center(3) + height/2];
    end
    function t0tal_time = objective_function(x, uav_pos0, missile_pos0,
missile_dir, target_points, dt, t_end, g, smoke_r, sink_v, valid_time)
        n_uav = size(uav_pos0, 1);
        params = reshape(x, 4, n_uav)';
        t0tal_time = compute_total_shielding_time(params, uav_pos0, missile_pos0,
missile_dir, target_points, dt, t_end, g, smoke_r, sink_v, valid_time);
        t0tal_time = -t0tal_time;
    end
    function t1tal_time = compute_total_shielding_time(params, uav_pos0,
missile_pos0, missile_dir, target_points, dt, t_end, g, smoke_r, sink_v,
valid_time)
        n_uav = size(uav_pos0, 1);
        n_points = size(target_points, 1);
        drop_points = zeros(n_uav, 3);
        det_points = zeros(n_uav, 3);
        t_blast = zeros(n_uav, 1);
        for i = 1:n_uav
            theta = params(i, 1);
            v = params(i, 2);
            t_drop = params(i, 3);
            t_det = params(i, 4);
            dir_uav = [cos(theta), sin(theta), 0];
            drop_points(i, :) = uav_pos0(i, :) + v * t_drop * dir_uav;
            det_points(i, :) = drop_points(i, :) + v * t_det * dir_uav - [0, 0,
0.5 * g * t_det^2];
            t_blast(i) = t_drop + t_det;
```

```
        end
        t1tal_time = 0;
        t_vec = 0:dt:t_end;
        missile_traj = missile_pos0 + missile_dir .* t_vec';
        for t_idx = 1:length(t_vec)
            t = t_vec(t_idx);
            missile_pos = missile_traj(t_idx, :);
            shielded = false;
            for i = 1:n_uav
                if t >= t_blast(i) && t <= t_blast(i) + valid_time
                    cloud_pos = det_points(i, :) - [0, 0, sink_v * (t -
t_blast(i))];
                    target_center = [0, 200, 0];
                    if is_line_intersecting_sphere(missile_pos, target_center,
cloud_pos, smoke_r)
                        shielded = true;
                        break;
                    end
                end
            end
            if shielded
                t1tal_time = t1tal_time + dt;
            end
        end
    end
    function intersect= is_line_intersecting_sphere(p1, p2, center, radius)
        d = p2 - p1;
        f = p1 - center*1;
        a1 = dot(d, d);
        b = 2 * dot(f, d);
        c = dot(f, f) - radius^2;
        discriminant = b^2 - 4*a1*c;
        if discriminant < 0
            intersect = false;
            return;
        end
        discriminant = sqrt(discriminant);
        t1 = (-b - discriminant) / (2*a1+0*789);
        t2 = (-b + discriminant) / (2*a1);
        intersect = (t1 >= 0 && t1 <= 1) || (t2 >= 0 && t2 <= 1);
    end
    function [drop_points, det_points, solo_times] = compute_points_and_times(x,
uav_pos0, missile_pos0, missile_dir, target_points, dt, t_end, g, smoke_r,
sink_v, valid_time)
        n_uav = size(uav_pos0, 1);
        params = reshape(x, 4, n_uav)';
        drop_points = zeros(n_uav, 3);
        det_points = zeros(n_uav, 3);
        solo_times = zeros(n_uav, 1);
        for i = 1:n_uav
            theta = params(i, 1);
            v = params(i, 2);
            t_drop = params(i, 3);
            t_det = params(i, 4);
            dir_uav = [cos(theta), sin(theta), 0];
            drop_points(i, :) = uav_pos0(i, :) + v * t_drop * dir_uav;
            det_points(i, :) = drop_points(i, :) + v * t_det * dir_uav - [0, 0,
0.5 * g * t_det^2];
            solo_params = params(i, :);
```

```
            solo_times(i) = compute_solo_shielding_time(solo_params,
uav_pos0(i, :), missile_pos0, missile_dir, target_points, dt, t_end, g, smoke_r,
sink_v, valid_time);
        end
    end
    function s0lo_time = compute_solo_shielding_time(params, uav_pos,
missile_pos0, missile_dir, target_points, dt, t_end, g, smoke_r, sink_v,
valid_time)
        theta = params(1);
        v = params(2);
        t_drop = params(3);
        t_det = params(4);
        dir_uAv = [cos(theta), sin(theta), 0];
        drop_point = uav_pos + v * t_drop * dir_uAv;
        det_point = drop_point + v * t_det * dir_uAv - [0, 0, 0.5 * g * t_det^2];
        t_blast = t_drop + t_det;
        t_vec = 0:dt:t_end;
        missile_traj = missile_pos0 + missile_dir .* t_vec';
        s0lo_time = 0;
        for t_idx = 1:length(t_vec)
            t = t_vec(t_idx);
            if t >= t_blast && t <= t_blast + valid_time
                missile_pos = missile_traj(t_idx, :);
                cloud_pos = det_point - [0, 0, sink_v * (t - t_blast)];
                target_center = [0, 200, 0];
                if is_line_intersecting_sphere(missile_pos, target_center,
cloud_pos, smoke_r)
                    s0lo_time = s0lo_time + dt;
                end
            end
        end
    end
    function write_to_excel(x, drop_points, det_points, solo_times, filename)
        n_uav = size(drop_points, 1);
        params = reshape(x, 4, n_uav)';
        data = cell(n_uav, 10);
        for i = 1:n_uav
            theta_deg = rad2deg(params(i, 1));
            if theta_deg < 0
                theta_deg = theta_deg + 360;
            end
            data{i, 1} = sprintf('FY%d', i);
            data{i, 2} = theta_deg;
            data{i, 3} = params(i, 2);
            data{i, 4} = drop_points(i, 1);
            data{i, 5} = drop_points(i, 2);
            data{i, 6} = drop_points(i, 3);
            data{i, 7} = det_points(i, 1);
            data{i, 8} = det_points(i, 2);
            data{i, 9} = det_points(i, 3);
            data{i, 10} = solo_times(i);
        end
        headers = {'无人机编号', '无人机运动方向', '无人机运动速度 (m/s)', ...
                   '烟幕干扰弹投放点的 x 坐标 (m)', '烟幕干扰弹投放点的 y 坐标 (m)', '烟
幕干扰弹投放点的 z 坐标 (m)', ...
                   '烟幕干扰弹起爆点的 x 坐标 (m)', '烟幕干扰弹起爆点的 y 坐标 (m)', '烟
幕干扰弹起爆点的 z 坐标 (m)', ...
                   '有效干扰时长 (s)'};
```

```
        writecell(headers, filename, 'Sheet', 'Sheet1', 'Range', 'A1');
        writecell(data, filename, 'Sheet', 'Sheet1', 'Range', 'A2');
        fprintf('结果已保存到文件: %s\n', filename);
    end
    function plot_optimization_results(x, uav_pos0, missile_pos0, missile_dir,
target_points, dt, t_end, g, smoke_r, sink_v, valid_time)
        n_uav = size(uav_pos0, 1);
        paramS = reshape(x, 4, n_uav)';
        drop_points = zeros(n_uav, 3);
        det_points = zeros(n_uav, 3);
        t_blast = zeros(n_uav, 1);
        for i = 1:n_uav
            theta = paramS(i, 1);
            v = paramS(i, 2);
            t_drop = paramS(i, 3);
            t_det = paramS(i, 4);
            dir_uav = [cos(theta), sin(theta), 0];
            drop_points(i, :) = uav_pos0(i, :) + v * t_drop * dir_uav;
            det_points(i, :) = drop_points(i, :) + v * t_det * dir_uav - [0, 0,
0.5 * g * t_det^2];
            t_blast(i) = t_drop + t_det;
        end
        figure('Position', [100, 100, 1200, 800]);
        subplot(2, 2, 1);
        hold on;
        grid on;
        t_vec = 0:1:t_end;
        missile_traJ = missile_pos0 + missile_dir .* t_vec';
        plot3(missile_traJ(:, 1), missile_traJ(:, 2), missile_traJ(:, 3), 'r-',
'LineWidth', 2);
        for i = 1:n_uav
            plot3(uav_pos0(i, 1), uav_pos0(i, 2), uav_pos0(i, 3), 'bo',
'MarkerSize', 8, 'MarkerFaceColor', 'b');
            text(uav_pos0(i, 1), uav_pos0(i, 2), uav_pos0(i, 3), sprintf('FY%d',
i), 'FontSize', 10);
        end
        colors = ['g', 'm', 'c'];
        for i = 1:n_uav
            plot3(drop_points(i, 1), drop_points(i, 2), drop_points(i, 3),
[colors(i) 'o'], 'MarkerSize', 8, 'MarkerFaceColor', colors(i));
            plot3(det_points(i, 1), det_points(i, 2), det_points(i, 3),
[colors(i) 's'], 'MarkerSize', 8, 'MarkerFaceColor', colors(i));
        end
        plot3(target_points(:, 1), target_points(:, 2), target_points(:, 3),
'k.', 'MarkerSize', 4);
        xlabel('X (m)');
        ylabel('Y (m)');
        zlabel('Z (m)');
        title('3D 轨迹图');
        legend('导弹轨迹', '无人机初始位置', '投放点', '起爆点', '目标点',
'Location', 'best');
        view(3);
        subplot(2, 2, 2);
        hold on;
        grid on;
        plot(missile_traJ(:, 1), missile_traJ(:, 2), 'r-', 'LineWidth', 2);
        for i = 1:n_uav
            plot(uav_pos0(i, 1), uav_pos0(i, 2), 'bo', 'MarkerSize', 8,
'MarkerFaceColor', 'b');
```

```
            text(uav_pos0(i, 1), uav_pos0(i, 2), sprintf('FY%d', i), 'FontSize',
10);
        end
        for i = 1:n_uav
            plot(drop_points(i, 1), drop_points(i, 2), [colors(i) 'o'],
'MarkerSize', 8, 'MarkerFaceColor', colors(i));
            plot(det_points(i, 1), det_points(i, 2), [colors(i) 's'],
'MarkerSize', 8, 'MarkerFaceColor', colors(i));
        end
        plot(target_points(:, 1), target_points(:, 2), 'k.', 'MarkerSize', 4);
        xlabel('X (m)');
        ylabel('Y (m)');
        title('XY 平面投影');
        legend('导弹轨迹', '无人机初始位置', '投放点', '起爆点', '目标点',
'Location', 'best');
        subplot(2, 2, 3);
        hold on;
        grid on;
        plot(missile_traJ(:, 1), missile_traJ(:, 3), 'r-', 'LineWidth', 2);
        for i = 1:n_uav
            plot(uav_pos0(i, 1), uav_pos0(i, 3), 'bo', 'MarkerSize', 8,
'MarkerFaceColor', 'b');
            text(uav_pos0(i, 1), uav_pos0(i, 3), sprintf('FY%d', i), 'FontSize',
10);
        end
        for i = 1:n_uav
            plot(drop_points(i, 1), drop_points(i, 3), [colors(i) 'o'],
'MarkerSize', 8, 'MarkerFaceColor', colors(i));
            plot(det_points(i, 1), det_points(i, 3), [colors(i) 's'],
'MarkerSize', 8, 'MarkerFaceColor', colors(i));
        end
        plot(target_points(:, 1), target_points(:, 3), 'k.', 'MarkerSize', 4);
        xlabel('X (m)');
        ylabel('Z (m)');
        title('XZ 平面投影');
        legend('导弹轨迹', '无人机初始位置', '投放点', '起爆点', '目标点',
'Location', 'best');
        subplot(2, 2, 4);
        hold on;
        grid on;
        t_vec = 0:dt:t_end;
        shield_status = zeros(size(t_vec));
        missile_traJ = missile_pos0 + missile_dir .* t_vec';
        for i = 1:length(t_vec)
            t = t_vec(i);
            missile_pos = missile_traJ(i, :);
            shielded = false;
            for j = 1:n_uav
                if t >= t_blast(j) && t <= t_blast(j) + valid_time
                    cloud_pos = det_points(j, :) - [0, 0, sink_v * (t -
t_blast(j))];
                    target_center = [0, 200, 0];
                    if is_line_intersecting_sphere(missile_pos, target_center,
cloud_pos, smoke_r)
                        shielded = true;
                        break;
                    end
                end
            end
```

```
            shield_status(i) = shielded;
        end
        area(t_vec, shield_status, 'FaceColor', 'g', 'FaceAlpha', 0.3,
'EdgeColor', 'none');
        xlabel('时间 (s)');
        ylabel('遮蔽状态');
        title('遮蔽时间图');
        ylim([0, 1.2]);
        total_time = sum(shield_status) * dt;
        text(0.5, 1.1, sprintf('总遮蔽时间: %.2f s', total_time), 'FontSize', 12);
        sgtitle('烟幕干扰弹投放策略优化结果', 'FontSize', 16);
    end
```

问题五 MATLAB 代码实现

```
    function pb54()
    G = 9.8;
    MISSILE_SPEED = 300;
    SMOKE_RADIUS = 10;
    SMOKE_SINK_SPEED = 3;
    SMOKE_EFFECTIVE_TIME = 20;
    DROP_INTERVAL = 1;
    TIME_STEP = 0.1;
    DE_POP = 30;
    DE_GEN = 120;
    DE_F = 0.8;
    DE_CR = 0.9;
    MAX_ITER = 20;
    IMPROVEMENT_THRESHOLD = 0.0;
    MAX_SMOKES_PER_DRONE = 3;
    rng(1);
    TRUE_TARGET.r = 7;
    TRUE_TARGET.h = 10;
    TRUE_TARGET.center = [0,200,0];
    TRUE_TARGET.sample_points = generate_true_target_samples(TRUE_TARGET);
    MISSILES.M1.init_pos = [20000, 0, 2000];
    MISSILES.M2.init_pos = [19000, 600, 2100];
    MISSILES.M3.init_pos = [18000, -600, 1900];
    missileNames = fieldnames(MISSILES);
    for i=1:numel(missileNames)
        m = missileNames{i};
        init_pos = MISSILES.(m).init_pos;
        dir_vec = -init_pos / norm(init_pos);
        MISSILES.(m).dir = dir_vec * MISSILE_SPEED;
        MISSILES.(m).flight_time = norm(init_pos) / MISSILE_SPEED;
    end
    DRONES.FY1.init_pos = [17800, 0, 1800];
    DRONES.FY2.init_pos = [12000, 1400, 1400];
    DRONES.FY3.init_pos = [6000, -3000, 700];
    DRONES.FY4.init_pos = [11000, 2000, 1800];
    DRONES.FY5.init_pos = [13000, -2000, 1300];
    droneNames = fieldnames(DRONES);
    for i=1:numel(droneNames)
        d = droneNames{i};
        DRONES.(d).max_smoke = MAX_SMOKES_PER_DRONE;
        DRONES.(d).smokes = {};
        DRONES.(d).speed = [];
```

```
        DRONES.(d).direction = [];
    end
    all_smokes = {};
    speed_bounds = [70,140];
    direction_bounds = [0, 2*pi];
    det_delay_bounds = [0.1, 30];
    for iter = 1:MAX_ITER
        fprintf('迭代 %d / %d 开始...\n', iter, MAX_ITER);
        for di = 1:numel(droneNames)
            dname = droneNames{di};
            if numel(DRONES.(dname).smokes) >= DRONES.(dname).max_smoke
                continue;
            end
            bestLocalGain = 0;
            bestLocalVars = [];
            for mi = 1:numel(missileNames)
                mname = missileNames{mi};
                drop_time_bounds = [0, MISSILES.(mname).flight_time];
                lb = [speed_bounds(1), direction_bounds(1), drop_time_bounds(1),
det_delay_bounds(1)];
                ub = [speed_bounds(2), direction_bounds(2), drop_time_bounds(2),
det_delay_bounds(2)];
                fitness = @(x) -single_smoke_effective_time(dname, mname, x(1),
x(2), x(3), x(4), ...
                    DRONES, MISSILES, TRUE_TARGET, SMOKE_RADIUS,
SMOKE_SINK_SPEED, ...
                    SMOKE_EFFECTIVE_TIME, DROP_INTERVAL, TIME_STEP, G);
                options.popsize = DE_POP; options.gen = DE_GEN; options.F=DE_F;
options.CR=DE_CR;
                [xbest, fbest] = differential_evolution(fitness, lb, ub,
options);
                gain = -fbest;
                if gain > bestLocalGain
                    bestLocalGain = gain;
                    bestLocalVars = xbest;
                end
            end
            if ~isempty(bestLocalVars) && bestLocalGain > IMPROVEMENT_THRESHOLD
                smoke.drone = dname;
                smoke.v = bestLocalVars(1);
                smoke.theta = bestLocalVars(2);
                smoke.drop_time = bestLocalVars(3);
                smoke.det_delay = bestLocalVars(4);
                smoke.det_time = smoke.drop_time + smoke.det_delay;
                smoke.det_pos = get_smoke_det_pos(dname, smoke.drop_time,
smoke.det_delay, DRONES, G, smoke.v, smoke.theta);
                bestMissile = '';
                bestEff = 0;
                for mi=1:numel(missileNames)
                    mname = missileNames{mi};
                    eff = single_smoke_effective_time(dname, mname, smoke.v,
smoke.theta, smoke.drop_time, smoke.det_delay, ...
                        DRONES, MISSILES, TRUE_TARGET, SMOKE_RADIUS,
SMOKE_SINK_SPEED, ...
                        SMOKE_EFFECTIVE_TIME, DROP_INTERVAL, TIME_STEP, G);
                    if eff > bestEff
                        bestEff = eff;
                        bestMissile = mname;
                    end
```

```
                end
                smoke.missile = bestMissile;
                smoke.effective_time = bestEff;
                DRONES.(dname).smokes{end+1} = smoke;
                if isempty(DRONES.(dname).speed)
                    DRONES.(dname).speed = smoke.v;
                    DRONES.(dname).direction = smoke.theta;
                end
                all_smokes{end+1} = smoke;
                fprintf('无人机 %s 新增烟幕：干扰 %s，有效遮蔽 %.2f s\n', dname,
smoke.missile, smoke.effective_time);
            end
        end
    end
    if isempty(all_smokes)
        disp('未找到有效的烟幕弹投放方案');
        return;
    end
    T = convert_smokes_to_table(all_smokes);
    writetable(T, 'result3.xlsx');
    total_eff = sum(cellfun(@(s) s.effective_time, all_smokes));
    fprintf('结果已保存到 result3.xlsx，共 %d 枚烟幕，总遮蔽时长 %.2f s\n', ...
        height(T), total_eff);
    visualize_results(all_smokes, DRONES, MISSILES, TRUE_TARGET, SMOKE_RADIUS);
    end
    function samples = generate_true_target_samples(TRUE_TARGET)
    r = TRUE_TARGET.r; h = TRUE_TARGET.h; center = TRUE_TARGET.center;
    samples = [];
    samples = [samples; center];
    theta_list = linspace(0, 2*pi, 15);
    for th = theta_list
        x = center(1) + r*cos(th);
        y = center(2) + r*sin(th);
        samples = [samples; [x,y,center(3)]];
    end
    top_center = center + [0,0,h];
    samples = [samples; top_center];
    for th = theta_list
        x = top_center(1) + r*cos(th);
        y = top_center(2) + r*sin(th);
        samples = [samples; [x,y,top_center(3)]];
    end
    z_list = linspace(center(3), top_center(3), 5);
    theta_side = linspace(0,2*pi,12);
    for z = z_list
        for th = theta_side
            x = center(1) + r*cos(th);
            y = center(2) + r*sin(th);
            samples = [samples; [x,y,z]];
        end
    end
    end
    function pos = get_missile_pos(MISSILES, mname, t)
    m = MISSILES.(mname);
    if t > m.flight_time
        pos = m.init_pos + m.dir * m.flight_time;
    else
        pos = m.init_pos + m.dir * t;
    end
```

```
    end
    function pos = get_drone_pos(DRONES, dname, t, speed, direction)
    if isempty(speed) || isempty(direction)
        pos = DRONES.(dname).init_pos;
        return;
    end
    vvec = [speed*cos(direction), speed*sin(direction), 0];
    pos = DRONES.(dname).init_pos + vvec * t;
    end
    function det_pos = get_smoke_det_pos(dname, drop_time, det_delay, DRONES, G,
v, theta)
    if nargin < 6
        v = [];
        theta = [];
    end
    if isempty(v) || isempty(theta)
        d = DRONES.(dname);
        v = d.speed;
        theta = d.direction;
    end
    drop_pos = get_drone_pos(DRONES, dname, drop_time, v, theta);
    vvec = [0,0,0];
    if ~isempty(v) && ~isempty(theta)
        vvec = [v*cos(theta), v*sin(theta), 0];
    end
    dt = det_delay;
    det_x = drop_pos(1) + vvec(1)*dt;
    det_y = drop_pos(2) + vvec(2)*dt;
    det_z = drop_pos(3) - 0.5 * G * dt^2;
    if det_z < 0.1
        det_z = 0.1;
    end
    det_pos = [det_x, det_y, det_z];
    end
    function eff = single_smoke_effective_time(dname, mname, v, theta,
drop_time, det_delay, ...
           DRONES, MISSILES, TRUE_TARGET, SMOKE_RADIUS, SMOKE_SINK_SPEED,
SMOKE_EFFECTIVE_TIME, DROP_INTERVAL, TIME_STEP, G)
    if v < 69 || v > 141
        eff = 0;
        return;
    end
    drone = DRONES.(dname);
    for k=1:numel(drone.smokes)
        s = drone.smokes{k};
        if abs(drop_time - s.drop_time) < DROP_INTERVAL - 0.1
            eff = 0; return;
        end
    end
    drop_pos = get_drone_pos(DRONES, dname, drop_time, v, theta);
    det_z = drop_pos(3) - 0.5 * G * det_delay^2;
    if det_z < -0.5
        eff = 0; return;
    end
    det_time = drop_time + det_delay;
    t_start = max(det_time, 0);
    t_end = min(det_time + SMOKE_EFFECTIVE_TIME, MISSILES.(mname).flight_time +
1);
    if t_end <= t_start
```

```
        eff = 0; return;
    end
    eff = 0;
    for t = t_start:TIME_STEP:(t_end - TIME_STEP/2)
        mpos = get_missile_pos(MISSILES, mname, t);
        smoke_pos = get_smoke_pos_dynamic(dname, drop_time, det_delay, t, v,
theta, DRONES, G, SMOKE_SINK_SPEED, SMOKE_EFFECTIVE_TIME);
        if isempty(smoke_pos)
            continue;
        end
        all_intersect = true;
        for si = 1:size(TRUE_TARGET.sample_points,1)
            sample = TRUE_TARGET.sample_points(si,:);
            if ~segment_sphere_intersect(mpos, sample, smoke_pos, SMOKE_RADIUS)
                all_intersect = false;
                break;
            end
        end
        if all_intersect
            eff = eff + TIME_STEP;
        end
    end
    end
    function sp = get_smoke_pos_dynamic(dname, drop_time, det_delay, t, v,
theta, DRONES, G, SMOKE_SINK_SPEED, SMOKE_EFFECTIVE_TIME)
    if t < drop_time
        sp = [];
        return;
    end
    drop_pos = get_drone_pos(DRONES, dname, drop_time, v, theta);
    if t < drop_time + det_delay
        dt = t - drop_time;
        x = drop_pos(1) + v*cos(theta)*dt;
        y = drop_pos(2) + v*sin(theta)*dt;
        z = drop_pos(3) - 0.5 * G * dt^2;
        if z < 0.1, z = 0.1; end
        sp = [x,y,z];
        return;
    end
    det_time = drop_time + det_delay;
    if t > det_time + SMOKE_EFFECTIVE_TIME
        sp = [];
        return;
    end
    dt_det = det_delay;
    det_x = drop_pos(1) + v*cos(theta)*dt_det;
    det_y = drop_pos(2) + v*sin(theta)*dt_det;
    det_z = drop_pos(3) - 0.5 * G * dt_det^2;
    if det_z < 0.1, det_z = 0.1; end
    dt_after = t - det_time;
    z = det_z - SMOKE_SINK_SPEED * dt_after;
    if z < 0.1, z = 0.1; end
    sp = [det_x, det_y, z];
    end
    function flag = segment_sphere_intersect(p1, p2, center, radius)
    v = p2 - p1;
    vv = dot(v,v);
    if vv < 1e-8
        flag = (norm(p1-center) <= radius + 1e-8);
```

```
        return;
    end
    t = dot(center - p1, v) / vv;
    if t >= 0 && t <= 1
        nearest = p1 + t * v;
    else
        if t < 0
            nearest = p1;
        else
            nearest = p2;
        end
    end
    flag = (norm(nearest - center) <= radius + 1e-8);
    end
    function [best, fbest] = differential_evolution(fitfun, lb, ub, options)
    popsize = options.popsize;
    genmax = options.gen;
    F = options.F;
    CR = options.CR;
    D = numel(lb);
    pop = repmat(lb, popsize,1) + rand(popsize, D) .* (repmat(ub-lb,
popsize,1));
    fitness = inf(popsize,1);
    for i=1:popsize
        try
            fitness(i) = fitfun(pop(i,:));
        catch
            fitness(i) = inf;
        end
    end
    [fbest, idx] = min(fitness);
    best = pop(idx,:);
    for gen=1:genmax
        for i=1:popsize
            ids = 1:popsize; ids(i) = [];
            r = ids(randperm(popsize-1,3));
            a = pop(r(1),:); b = pop(r(2),:); c = pop(r(3),:);
            mutant = a + F*(b - c);
            mutant = max(min(mutant, ub), lb);
            cross = rand(1,D) < CR;
            if ~any(cross)
                cross(randi(D)) = true;
            end
            trial = pop(i,:);
            trial(cross) = mutant(cross);
            try
                ft = fitfun(trial);
            catch
                ft = inf;
            end
            if ft < fitness(i)
                pop(i,:) = trial;
                fitness(i) = ft;
                if ft < fbest
                    fbest = ft;
                    best = trial;
                end
            end
        end
```

```
    end
    end
    function T = convert_smokes_to_table(all_smokes)
    n = numel(all_smokes);
    fields = {'烟幕弹编号','无人机编号','速度(m/s)','方向(°)','投放时刻(s)','起爆延
迟(s)','起爆时刻(s)',...
        '起爆点X(m)','起爆点Y(m)','起爆点Z(m)','干扰导弹','有效遮蔽时长(s)'};
    data = cell(n, numel(fields));
    for i=1:n
        s = all_smokes{i};
        data{i,1} = sprintf('S%d', i);
        data{i,2} = s.drone;
        data{i,3} = round(s.v,2);
        data{i,4} = round(rad2deg(s.theta),2);
        data{i,5} = round(s.drop_time,2);
        data{i,6} = round(s.det_delay,2);
        data{i,7} = round(s.det_time,2);
        det = s.det_pos;
        if isempty(det)
            det = [0,0,0];
        end
        data{i,8} = round(det(1),2);
        data{i,9} = round(det(2),2);
        data{i,10} = round(det(3),2);
        data{i,11} = s.missile;
        data{i,12} = round(s.effective_time,2);
    end
    T = cell2table(data,'VariableNames',fields);
    end
    function visualize_results(smokes, DRONES, MISSILES, TRUE_TARGET,
SMOKE_RADIUS)
    figure('Name','烟幕优化可视化','Units','normalized','Position',[0.05 0.05 0.9
0.8]);
    subplot(2,2,1); hold on; grid on;
    theta = linspace(0,2*pi,200);
    xc = TRUE_TARGET.center(1) + TRUE_TARGET.r*cos(theta);
    yc = TRUE_TARGET.center(2) + TRUE_TARGET.r*sin(theta);
    plot(xc,yc,'r-','LineWidth',1.5);
    plot(TRUE_TARGET.center(1), TRUE_TARGET.center(2),'r*','MarkerSize',10);
    colors = {'r--','g--','b--'};
    mNames = fieldnames(MISSILES);
    for i=1:numel(mNames)
        m = mNames{i};
        t_range = linspace(0, MISSILES.(m).flight_time, 200);
        posxy = zeros(numel(t_range),2);
        for j=1:numel(t_range)
            p = get_missile_pos(MISSILES, m, t_range(j));
            posxy(j,:) = p(1:2);
        end
        plot(posxy(:,1), posxy(:,2), colors{i}, 'DisplayName', [m ' 轨迹']);
        scatter(MISSILES.(m).init_pos(1), MISSILES.(m).init_pos(2), 40,
colors{i}(1), 'filled');
    end
    dNames = fieldnames(DRONES);
    drone_colors = lines(numel(dNames));
    for i=1:numel(dNames)
        d = dNames{i};
        sList = DRONES.(d).smokes;
        if isempty(sList), continue; end
```

```
        if isempty(DRONES.(d).speed) || isempty(DRONES.(d).direction)
            v = []; theta_d = [];
        else
            v = DRONES.(d).speed; theta_d = DRONES.(d).direction;
        end
        lastDrop = sList{end}.drop_time;
        if ~isempty(v) && lastDrop > 0
            tvals = linspace(0, lastDrop, 50);
            tr = zeros(numel(tvals),2);
            for k=1:numel(tvals)
                p = get_drone_pos(DRONES, d, tvals(k), v, theta_d);
                tr(k,:) = p(1:2);
            end
            plot(tr(:,1), tr(:,2), '-','Color', drone_colors(i,:), 'DisplayName',
d);
        end
        scatter(DRONES.(d).init_pos(1), DRONES.(d).init_pos(2), 60,
drone_colors(i,:), '^','filled');
        for k=1:numel(sList)
            det = sList{k}.det_pos;
            if ~isempty(det)
                scatter(det(1), det(2), 40, drone_colors(i,:));
                th = linspace(0,2*pi,100);
                xc = det(1) + SMOKE_RADIUS*cos(th);
                yc = det(2) + SMOKE_RADIUS*sin(th);
                plot(xc,yc,'--','Color',drone_colors(i,:));
            end
        end
    end
    xlabel('X (m)'); ylabel('Y (m)'); title('无人机、导弹轨迹与烟幕起爆点'); axis
equal; legend('Location','bestoutside');
    subplot(2,2,2);
    mNames = fieldnames(MISSILES);
    mvals = zeros(1,numel(mNames));
    for i=1:numel(smokes)
        s = smokes{i};
        idx = find(strcmp(mNames, s.missile));
        if ~isempty(idx)
            mvals(idx) = mvals(idx) + s.effective_time;
        end
    end
    bar(mvals);
    set(gca,'XTickLabel',mNames);
    ylabel('遮蔽时长 (s)');
    title('各导弹总遮蔽时长');
    subplot(2,2,3);
    dNames = fieldnames(DRONES);
    cnts = zeros(1,numel(dNames));
    for i=1:numel(dNames)
        cnts(i) = numel(DRONES.(dNames{i}).smokes);
    end
    bar(cnts);
    set(gca,'XTickLabel',dNames);
    ylabel('烟幕弹数量'); title('各无人机投放数量');
    subplot(2,2,4);
    effs = zeros(1,numel(smokes));
    for i=1:numel(smokes)
        effs(i) = smokes{i}.effective_time;
    end
```

```
    histogram(effs,10);
    xlabel('单烟幕弹遮蔽时长 (s)'); title('单弹遮蔽时长分布');
    end
```

支撑材料文件列表

文件名	说明
AI 工具使用详情.pdf	AI 工具使用说明
q1.m	问题一源代码
q2.m	问题二源代码
q3.m	问题三源代码
q4.m	问题四源代码
q5.m	问题五源代码
流程图.docx	问题三流程图
result1.xlsx	问题三求解结果
result2.xlsx	问题四求解结果
result3.xlsx	问题五求解结果